LayerFS: A Recursive Filesystem for Ephemeral Computing in AI Agents

Draft — authors withheld

Abstract

AI coding agents spend most of their time thinking, not running: in 2,095 agent sessions we analyzed, the model takes a median 84% of active time, and most tool calls only touch files. Yet each agent is given a whole computer for its task, and that computer's disk is the only copy of its work. LayerFS inverts the arrangement with *ephemeral computing*: compute is borrowed per command, and all state lives outside it, in one shared, versioned Store. File calls run in the agent's own process against it and need no computer. Each shell call mounts a complete, writable Workspace—source, `.git`, installed dependencies, build outputs and every ignored file—over an immutable committed root, in time independent of tree size; it commits what changed and goes away. Every step becomes an immutable commit, and the filesystem is *recursive*: any commit, including one inside a fork of a fork, can start a new Branch at the same constant cost, without copying data.

Recording whole trees at every step is affordable because commits share structure. In a 13-hour build by nine agents, 4,354 commits of a 9 GB tree on 211 Branches occupy 4.93 GB—less than half of one checkout of the final tree—and the median edit commit adds 0.9 KB. Mounts take a median 7 ms and forks 5 ms. We find that commit granularity decides which storage mechanism matters: per-call commits make exact deduplication dominant, while coarse snapshots rely on delta compression. Against ovnfs, an overlay filesystem for agent checkouts, LayerFS adds 6–22× fewer bytes per commit on code histories and commits 1.5–21× faster from a cold cache; it is slower on cold large-file reads, bulk import and brand-new data.

1 Introduction

An AI coding agent works in a loop: the model reads context and decides on an action, a tool executes it, and the result returns to the model. Most tool calls read or edit files; a minority run commands that need an operating system—building, testing, installing packages. In 2,095 agent sessions we analyzed, the model accounts for a median 84% of each session's active time, and about 63% of file and shell calls are file operations that need no process at all (§2.1).

Today's agent infrastructure nevertheless starts from the computer. Each task receives a sandbox—a container [1] or a micro-VM [5]—whose local filesystem holds the repository, its installed dependencies and every intermediate result. The sandbox must stay alive for the whole task because its disk *is* the agent's state. This couples three things that have different lifetimes: the agent's files (which must survive the task, be inspected, branched and resumed), the compute that runs a command (needed for seconds), and the model's thinking time (minutes, needing no compute at all). It also makes checkpointing, branching and parallel exploration expensive: copying or snapshotting a sandbox costs seconds and gigabytes, and Git-based alternatives drop the ignored files—`node_modules`, build outputs, caches—so every fresh checkout must reinstall before it can run anything.

We present LayerFS, a filesystem for **ephemeral computing**: compute is borrowed per command and given back, and state lives outside it, in durable history that every agent shares. Each agent mounts a complete, writable Workspace over shared committed history, does its work, commits what it changed, and goes away. LayerFS is also **recursive**: every step is a commit, and any commit—including one inside a fork of a fork, or in the middle of an earlier rollout—can start a new Branch at the same constant cost. The two halves, ephemeral Workspaces and durable shared history, have different jobs.

Ephemeral Workspaces are the compute side, borrowed per command:

- **File calls need no computer.** Reads, writes and edits run in the agent's process, directly against the Store; each write is one atomic commit (§4.1).
- **Shell calls get a Workspace.** A Workspace is a complete POSIX view of a Branch, including dependencies, build outputs and ignored files. Mounting scans, copies and installs nothing; its only new file is a small private overlay database (§4.2). After the command, a commit records every change regardless of which process made it, and unmount discards the view (§4.4).

Durable shared history is the state side. It outlives every Workspace, Sandbox and agent, and every agent, Branch and project on a Store shares it:

- **Every step is a commit** that can be read, diffed or blamed.
- **Forks are recursive.** Any commit, at any fork depth, can start a Branch by writing one metadata row, so rollouts, best-of-*N* sampling [9] and tree search [38, 17] become cheap filesystem operations; every step of a rollout is itself a node to search from (§5.1, §5.3).
- **Many agents can work on one Branch**, with conflicting writes refused atomically rather than silently interleaved (§5.2).

Recording everything must be nearly free, or none of

this is practical: every commit names a whole tree, build outputs included. LayerFS shares first and compresses second. Unchanged subtrees keep their identities, so a commit costs what changed; content addressing reuses identical files and chunks across steps, Branches and projects; and what remains new is delta-encoded against a base chosen from filesystem events such as rename-over saves, then compressed (§6).

The individual storage techniques—content-defined chunking [21, 36], content addressing [27, 2], resemblance-based delta compression [31, 39]—are well known; backup systems and version-control systems apply them *offline*, over complete snapshots. LayerFS applies them *online*, inside a live filesystem's commit path, at tool-call latency, and organizes agent execution around the result. Doing so exposes a finding we did not expect: *commit granularity decides which mechanism saves space*. With one commit per tool call, consecutive states are nearly identical and exact deduplication carries the load (1,179× against 5.4× from deltas and compression in a nine-agent run); with coarse snapshots the burden shifts to delta encoding.

This paper makes the following contributions:

- A characterization of where agent time and tool calls go, from 2,095 sessions across two agent systems (§2.1).
- The ephemeral-Workspace model: complete views whose mount cost is independent of tree size and whose commit cost follows the change, over recursive history in which any commit, at any fork depth, can be forked (§3–§5).
- A Store that builds and encodes immutable objects in parallel and serializes metadata publication through one writer with short holds, so many Workspaces and in-process callers share one database (§5.4).
- An online storage pipeline—structural sharing first, then delta bases chosen from filesystem events—and the observation that commit granularity decides which of the two matters (§6, §8.5).
- An evaluation including a 13-hour, nine-agent build and comparisons with an agent overlay filesystem, Btrfs snapshots and Git, including the cases where LayerFS loses (§8).

2 Agents Think More Than They Run

2.1 How agents use computers

To ground the design we analyzed agent runs recorded on one developer's machine: 483 sessions of an open-source agent harness driving six model families (49,757 agent steps, 251 hours inside steps), and 1,612 transcripts of a commercial coding agent (14,641 prompts, 1,043 active hours, with time spent waiting for the human excluded). Table 1 summarizes them.

Two observations follow. **Most time is spent in the model.** In aggregate the model accounts for 64–69% of active wall time, and in the median session for 84–85%; the median gap between a tool's result and the next tool call is about six seconds. A computer held for the task idles through all of it. **Most tool calls need no computer.** Agents issue most calls *as* shell commands—the coding agent chose its shell tool for 89% of file and shell calls even though it also offers file tools—but more than half of those commands only read or search files. Counting dedicated file tools, 62–63% of calls are file operations that a filesystem API can serve directly; about a third need to run a process. Figure 1 shows what this means for one turn: a computer is needed for a few short intervals, not for the whole task.

	Harness	Coding agent
Sessions / transcripts	483	1,612
Model share of active time, aggregate	64.3%	68.9%
per session, median	85%	84%
Tool result → next call, median	6.5 s	5.6 s
File + shell calls	61,001	211,613
file operations (any tool)	63.3%	61.9%
need process execution	33.5%	31.6%
Shell commands that only read/search	52.4%	53.5%
Shell calls per prompt, median / mean	3 / 21.8	2 / 12.9

Table 1: Where agent time and tool calls go. "File operations" counts dedicated file tools plus shell commands whose program reads, searches or moves files; the rest builds, tests, runs programs, writes Git state or manages infrastructure.

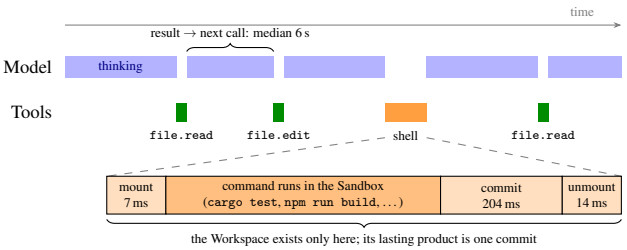


Figure 1: One agent turn, schematically. The model thinks most of the time (median 84–85% per session, Table 1). File calls write to the Store from the agent's process and need no computer. Only a shell call borrows a Workspace, for the length of one command; latencies are medians from the nine-agent build (Table 4).

These numbers come from one developer's systems-heavy work, which inflates build and test time, and we classify shell commands by their first program word. They are indicative, not general; §9 lists what a cleaner measurement would record.

2.2 The cost of a computer per agent

A sandbox held for a whole task pays for idle time while the model thinks, and it is the sole home of the agent's work: losing it loses the state, and branching it means copying it. Two common ways to avoid holding a sandbox both fall short.

Re-create from Git. Cloning or checking out a commit is cheap only because Git ignores the files that make a tree runnable: installed dependencies, build outputs and caches. A fresh checkout must reinstall before the first test can run,

a cost measured in minutes for typical JavaScript or Rust projects. Git also commits by scanning the working tree, and it is not a mountable filesystem.

Snapshot the sandbox. Container layers, VM snapshots and copy-on-write file systems capture everything, but at whole-file or whole-image granularity, or only for in-place updates. Overlay filesystems copy a file up in full on its first write [4]. Snapshot filesystems such as Btrfs [29] share extents that stay in place, but a file rewritten whole—as editors, Git and package managers do when they write a temporary file and rename it over the original—is new data.

2.3 Agent filesystems today

Several systems target agents directly. ovnfs [32] provides disposable, Git-integrated agent checkouts through file-level copy-on-write overlays with snapshots into a packed object store. AgentFS [33] stores a filesystem in SQLite as fixed 4 KiB chunk rows with a per-session delta database, but (in the version we studied, 0.6.4) keeps no version history or snapshots. JuiceFS [3] and earlier database-backed filesystems [24, 28] keep metadata in a database but are not versioned. None combines a complete mountable tree, per-call history and fork-from-anywhere at low storage cost.

2.4 Goals

1. **Complete views.** A Workspace is the whole tree, ignored files included, so commands run as in an installed checkout.
2. **Constant-cost bootstrap.** Mounting does not scan, copy or install; cost is independent of tree size.
3. **Commit cost follows the change**, not the size of the tree or of the edited file.
4. **Every step is history.** Every tool call that changes the tree is an immutable, forkable commit.
5. **Affordable history.** Storage grows with what changed, across steps, Branches and projects.
6. **Shared without a server.** Many processes and Workspaces share one Store with no database server in the data path.

3 Ephemeral Workspaces, Shared History

Objects. A *project* is one source tree. Its committed state lives in a *Store*: one SQLite database file in WAL mode, or one PostgreSQL schema, holding immutable content-addressed objects and the history records that name them. A *Branch* is a named line of *Commits*; each Commit names a filesystem root and its parent(s). The distinguished Branch `main` is read-only and grows by *Layers*, each a published state. Project initialization reads the source tree once, including `.git`, dependencies, caches, symlinks and every Git-ignored file, and makes it the first Layer.

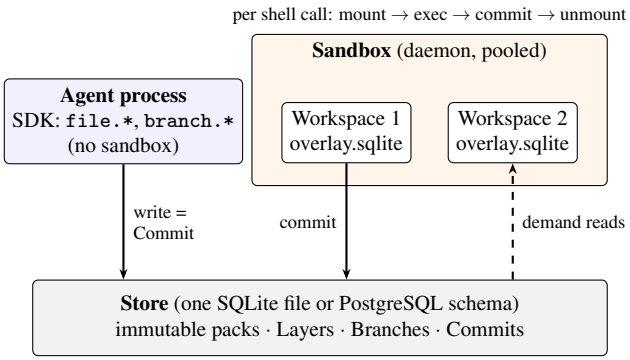


Figure 2: LayerFS architecture. State lives only in the Store. File calls run in the agent's process. Shell calls borrow a Workspace: a private overlay over a committed root, created at mount and deleted at unmount.

Two execution paths. Figure 2 shows the two ways an agent touches state. *File calls* (`file.read`, `file.write`, `file.edit`, and history calls such as `branch.fork` or `file.blame`) run inside the caller's process, which opens the Store directly. *Shell calls* run in a *Sandbox*: a long-lived daemon plus an isolation environment (Linux namespaces and cgroups inside a VM or container, or a host process on macOS). The orchestrator mounts a Branch as a Workspace in the Sandbox, runs the command, commits and unmounts.

What is ephemeral. The unit created and destroyed per call is the Workspace—a private overlay database plus a kernel mount—not the Sandbox. A Sandbox holds up to 16 Workspaces, shares an object cache among them, and can be discarded at any time: it never holds committed state, so losing it loses only uncommitted overlays, and any Sandbox attached to the Store can mount any Branch. This is what lets compute be pooled, replaced and scaled independently of the agents whose state it serves.

What is durable. History—every object, Commit, Branch and Layer—lives only in the Store and is never rewritten. By *durable* we mean that it outlives every Workspace, Sandbox and agent: losing any of them loses no committed work. We do not mean crash-durable in the `fsync` sense; the Store trades that for latency (§9).

Interface. One call table defines the whole interface: 73 calls in ten families, from which the `layerfs` command line and the Node and Python SDKs are generated, so `layerfs.file.write` in an SDK is `layerfs file write` in a shell. Table 2 groups the calls an agent uses by the role they play; Appendix H lists all of them.

Role	Calls
No computer: file calls, one Commit per write	`file.read`, `write`, `edit`, `apply`, `cp`, `mv`, `rm`, `mkdir`, `stat`, `list`, `glob`, `grep`
Borrowed compute: one shell call	`workspace.mount`, `exec`, `commit`, `unmount`; `workspace.status`, `exec_cancel`
Compute pool	`sandbox.create`, `close`, `get`, `list`; `machine.*`
Recursive forks	`branch.fork` (from any Branch, Commit or Layer), `merge`, `publish`
Every step a checkpoint	`commit.list`, `get`, `diff`; `branch.diff`, `commits`; `file.blame`, `revision`, `diff`; `file.read` at a Commit
Shared state	`store.create`, `connect` (PostgreSQL), `push`, `attach`; `project.init`, `export`
Measurement	`store.stats`, `project.stats`, `branch.commits -include-storage`

Table 2: The LayerFS 2.0 interface by role. A call written after a family name belongs to that family (`write` is `file.write`).

4 Ephemeral Workspaces

4.1 File calls without a computer

A file call binds a private Workspace that is never mounted. Its overlay is a copy of the process's template; the call applies each change as overlay jobs and then commits through exactly the same capture, construction, merge and publication path as a mounted Workspace (§4.4). A write is therefore one atomic operation and one Commit. A write that leaves the tree unchanged answers `UpToDate` and creates nothing. Because the caller opens the Store itself, there is no server in the data path.

Opening a SQLite Store from several processes requires one kernel and a local filesystem whose locking SQLite trusts (ext4, xfs or btrfs on Linux; APFS on macOS); a guard refuses anything else as `Unsupported`. Hosts that cannot open the Store directly forward each call once, with no retry or replay, to a loopback worker on the Store's machine. With the PostgreSQL backend, callers on different machines can open one Store, since PostgreSQL arbitrates access instead of the local kernel.

4.2 Constant-cost mount

Mounting a Branch performs four steps: read the Branch's head from the Store, decode and authenticate the root object, copy a pre-built, schema-only overlay template into a fresh SQLite file, and attach a FUSE session (Linux) or a loopback NFSv3 export (macOS). None of these steps depends on the size of the tree: nothing is scanned, copied, restored or installed. Directory pages, inode-table pages and file extents are fetched on demand from the Store when first accessed, authenticated against their object identifiers, and kept in a per-daemon cache keyed by identity and shared by all Workspaces of that daemon. The first read of a file still pays real I/O; what mount avoids is work proportional to the tree.

4.3 A private overlay per Workspace

Each mounted Workspace owns one SQLite overlay database configured for speed over durability (in-memory journal, `synchronous=OFF`, exclusive locking, in-memory temporary storage). It records only changes: names and whiteouts, inodes, and file bytes as 4 KiB cells and 32 KiB dense runs. *Reference pieces* describe ranges equal to already-committed bytes without copying them, so a small write to a large file stores only what was written. Open-file, directory-position and kernel-reference state live in daemon memory.

One lock guards a Workspace's connection. It is held for exactly one short atomic job—at most one transaction—and never across Store I/O, a command or a commit. Every job is bounded by a window (FUSE writes and reads of at most 128 KiB, 256 KiB transfers on the NFS route, 1,024-row pages), so no file, edit or Workspace size is capped by resident memory. Workspaces share no overlay connection, writer, page cache or memory budget; the only shared state is committed immutable content and the Store.

FUSE kernel write-back caching is disabled: every `write(2)` is published to the overlay before it returns. This makes capture exact—an acknowledged write is always in the next commit—at the price of writing new data through SQLite at write time (§8.8).

Execution is not the filesystem's business. Any process that can see the mount uses ordinary system calls. The daemon supervises no commands and registers no executions; the Sandbox runtime launches commands, owns their streams and exit status, and places each in its own cgroup. A commit captures what was published, not what a registered command did.

4.4 Commit cost follows the change

A commit proceeds in five steps.

1. **Capture.** The overlay's active generation is sealed as the published frontier and a new one opens. No rows are copied and nothing is filtered by process: changes from several commands, including background processes, are captured together.
2. **Construct.** One producer thread reads only the sealed rows and the immutable base. Unchanged directories keep their stored roots. A changed file applies localized edits to its extent tree: it splits stored extents at edit boundaries, chunks only the new bytes and rebuilds only the pages on the paths to the root (§6.3). A file recreated with identical bytes reuses its stored root.
3. **Merge.** If the Branch head moved since the Workspace's base, the candidate is merged three-way with the new head before publication.
4. **Publish.** A short *publication lane*, holding the Store's single writer, re-checks the head, merges whatever landed

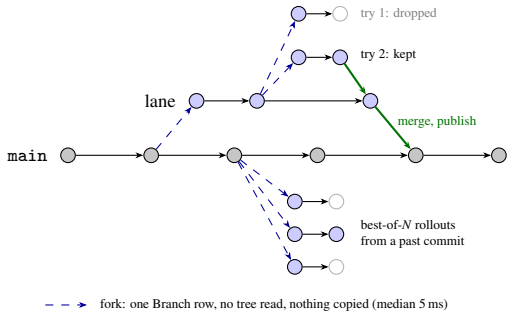


Figure 3: Recursive forks. Any Commit or Layer can start a Branch, including a past commit and a fork of a fork. A dropped fork costs only the bytes it changed; a kept one is merged back and published.

in the meantime (bounded) and advances the Branch in one transaction.

5. **Install.** The Workspace's base is rebound to the new root without a tree walk; changes made after the capture remain on top.

A commit with no changes answers `UpToDate` without capturing, saving or taking the lane.

Commit captures a filesystem, not a process. Unflushed dirty `mmap` pages and writes by still-running descendants that have not reached the overlay are not captured; command exit is not treated as proof that a command's descendants are finished.

4.5 Unmount and reclamation

Unmount refuses a Workspace with uncommitted changes unless told to discard them. It then detaches the kernel mount, drains in-flight requests, revokes kernel references and deletes the overlay file as a whole. Committed history is untouched. A Workspace is therefore a disposable view whose only lasting product is a commit.

5 Shared History

History is the half of LayerFS that lasts. Every agent, Branch and project on a Store shares one object graph and one record of Commits and Layers, so forking, merging and many agents working at once are operations on metadata.

5.1 Recursive forks

`branch.fork(name, from)` writes one Branch row naming its start: the newest Layer of `main`, another Branch's head, or any Commit or Layer by identifier. It reads no tree and copies no object, so a fork of a fork of a fork costs the same as the first, and its mount is the same constant-cost bind (Figure 3). This is what we mean by a *recursive* filesystem: a fork is a full peer of its parent, and forking it again costs nothing more. Changes a fork makes are deduplicated against everything already stored, so a losing fork costs only the bytes it changed.

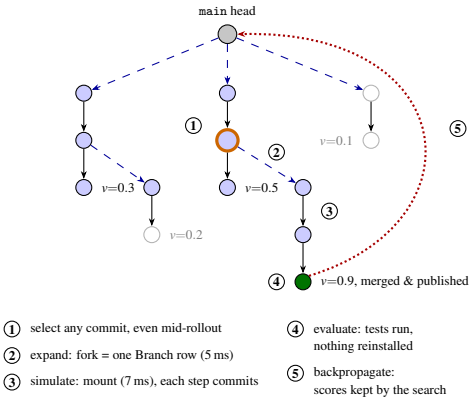


Figure 4: Monte Carlo tree search over commits. Every node is a Commit—a complete tree with dependencies and build outputs—and every rollout step is a node, so search can expand from the middle of an earlier rollout (orange). Dashed edges are forks, solid edges are steps; dropped Branches (white) cost only the bytes they changed. Schematic; values are illustrative.

A winning fork is folded back by `branch.merge` (a fast-forward when one side contains the other) and published as `main`'s next Layer.

5.2 Many agents, one Branch

Agents do not lock files. Each commit merges with the Branch head it finds when it publishes. Changes to different files land on top. A file changed on both sides is merged three-way: text is line-merged and overlapping hunks become a *conflict*; binaries keep the local side. A conflict never half-lands: nothing is published. In a Workspace, the merged tree with marker blocks is installed so the agent can resolve it with ordinary tools, and a commit is refused while a complete marker block remains.

Some files are rewritten wholesale by tools—lockfiles, build outputs, caches—and a line merge of them is meaningless. A `/.lww` file at the tree root lists gitignore-style patterns; for matching paths changed on both sides, the newer modification time wins with no conflict, and each decision is recorded on the commit.

5.3 Search over filesystem states

Because forks are recursive—one row at any depth—and every step is a commit, search algorithms that branch over environment states map directly onto LayerFS. Figure 4 shows Monte Carlo tree search [17, 10] over commits. A node is a Commit. Selection picks any Commit by identifier. Expansion forks it. A simulation mounts a Workspace on the new Branch and lets the agent act; each step commits, adding a median 0.9 KB for an edit in the nine-agent build. Evaluation runs the task's tests in the same Workspace without reinstalling anything, because ignored files are part of the state. Backpropagation updates statistics the search keeps

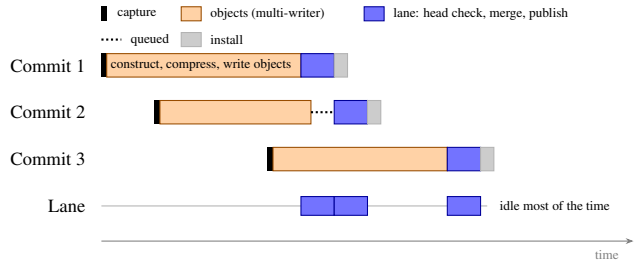


Figure 5: Three overlapping commits. Object construction runs in parallel, since immutable, content-addressed objects commute; on SQLite the pack writes themselves serialize briefly. Only the short metadata step takes the publication lane; commit 2 queues behind commit 1. Schematic.

outside the tree; LayerFS stores no scores. A dropped Branch costs only the bytes it changed, and the winner is merged and published.

Two properties distinguish this from search over sandbox snapshots. *Every rollout step is a resumable node*: a search can later expand from the middle of an earlier rollout without replaying it, whereas container or VM snapshots cost seconds and gigabytes and are taken only at chosen points. *Expansion cost does not depend on depth*: a fork of a fork writes one row and mounts in the same constant time as the first. Siblings that write identical files—the same dependency install, the same generated output—store them once, because deduplication spans every Branch on the Store. The same pattern serves best-of-*N* sampling [9], tree-of-thought search [38], reinforcement learning rollouts whose environment is a source tree, and agents that try a change to their own code in a fork before adopting it. These are consequences of the design rather than measured results; §9 lists the limits that wide searches would hit.

5.4 Parallel objects, single-writer metadata

The Store separates writes by whether they need an order. **Object data is multi-writer.** Objects are immutable and content-addressed: two writers that store the same object store the same bytes, so object writes commute and need no global order. Every commit, in any process, constructs, encodes, compresses and seals its packs on its own, concurrently with other commits. With PostgreSQL, pack bodies are copied into the Store before the lane row is locked. With SQLite, each pack batch is a short write transaction that also takes the Store's directory lock across processes, so the CPU work—construction, encoding, compression, sealing—runs in parallel while the batch writes serialize briefly. **Metadata is single-writer.** Order matters for the locator index that makes an object addressable—an identifier is inserted only if absent, which is unique because one writer inserts—and for history: Branch heads, Commits and Layers. These go through one writer at a time, enforced by an in-process FIFO turn and a directory lock across processes (SQLite) or a lane row (PostgreSQL). A commit holds this *publication lane* only for its head check, a bounded merge of whatever landed meanwhile, and the atomic publish; reads never take it (Figure 5). Writers queue for their turn instead of failing at once; a writer that cannot get it within a bounded wait is refused as `Busy`, and nothing is retried automatically. In the nine-agent run, 0.66% of writes were refused this way (§8.2).

This is the split used by lakehouse table formats, which write data files in parallel and serialize commits to a metadata log [6], and by Git, which writes objects freely and updates references atomically. We chose it over sharding or a multi-writer metadata engine because agents' tool calls are separated by model latency, so concurrent publications on one Store are uncommon: the uncontended commit is the case to optimize, and contention must only stay correct and bounded. Work that grows with tree size, history length or Branch count inside the lane is treated as a defect.

The SQLite Store runs in WAL mode with `synchronous=OFF`. It survives process failure but not a kernel or VM crash, a deliberate trade for latency that we discuss in §9.

6 Recording Everything: Share First, Then Compress

Every commit names a whole tree, dependencies and build outputs included, so recording everything is practical only if what did not change costs nothing. LayerFS stores committed state as a Merkle DAG of immutable, content-addressed objects. It *shares first*: unchanged subtrees, files and chunks keep their identities and are reused, across steps, Branches and projects. It *compresses second*: what is genuinely new is delta-encoded against one well-chosen base and compressed, inside the commit that creates it. Section 8.5 shows which of the two dominates depends on how often agents commit, and Appendix D traces the pipeline through 14 workload patterns and two worked examples; Appendix F bounds the time and storage each change costs and how storage grows over many snapshots.

6.1 Objects and identity

Each object is a role-tagged value in a fixed envelope. Its identifier is the BLAKE3 hash [23] of a domain string, `layerfs/object/v2\0`, followed by the canonical bytes. Thirteen roles form the tree: a filesystem root names a B^+-tree inode table keyed by a stable serial; an inode value names a content root and metadata; content is a whole-file object, a file state over a B^+-tree of extent pages referencing chunks, a directory B^+-tree from names to serials, or a symlink. Decoders accept exactly one encoding per value, and references are derived from bytes, never supplied by callers.

Physical representation—compression, delta base, pack location—is outside the identifier. Every read rebuilds canonical bytes and re-hashes them, so a corrupted or mis-decoded record is detected at read time.

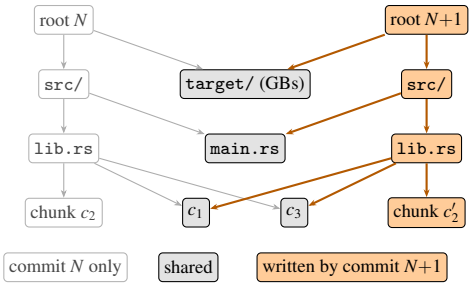


Figure 6: Structural sharing after a one-line edit to `lib.rs`. Commit $N+1$ writes only the path from the changed chunk to the root; every other object, including all of `target/`, is referenced, not copied. Simplified: directories, the inode table and large files are B^+-trees, so the new path is a few pages.

6.2 Structural sharing and exact reuse

A commit rebuilds only the paths from changed entries to the root. Unchanged directories keep their stored roots, an inode-table page that no change reaches is not rewritten, and an unchanged file keeps its content root, so committing a 9 GB tree after a one-line edit writes a few pages (Figure 6). Publishing `main`'s next Layer names an existing root rather than copying it.

Objects reach storage in batches (*waves*). Duplicates within a wave are compared byte for byte. An object already stored with the same identifier, role and length is reused without reading it back; integrity is deferred to the read path, which authenticates every decoded object. Concurrent writers are arbitrated by insert-if-absent locators, and a losing writer compares its bytes with the winner's. Reuse spans the whole Store: across steps, Branches and projects.

6.3 Chunking and localized edits

Files below a per-Store cutoff (128 KiB by default) are single whole-file objects. Larger files are split by a FastCDC-style [36] GEAR scanner with an 8 KiB minimum, 16 KiB normalization point and 32 KiB maximum. The chunking profile is hashed into every file state so the partition cannot drift silently. Runs of zeros exploit the scanner's 32 KiB period: interior zero chunks become a repeat count and identical mapping pages are reused, so construction work grows with tree height rather than hole length.

A *localized edit* splits stored extents at its boundaries, discards the replaced middle unread, chunks only the replacement bytes and rebuilds pages only on the join paths. Unchanged extents keep their identities, including slices of stored chunks, so the same overwrite reads 3 pages on an 8 MB file and 4 on a 24 MB file. The price is that equal bytes can have different roots depending on edit history; diff and merge therefore compare bytes (skipping shared extents) rather than roots.

6.4 Delta bases from filesystem events

Each new payload object (whole file or chunk) first receives an independent record: zstd [11] level 3, or raw when a 1 KiB probe predicts no gain. Selection then makes *at most one* delta trial: a zstd frame compressed with a single base payload as prefix. The delta is kept only if its full record, including the 32-byte base identifier, is strictly smaller.

The single base is chosen from *filesystem events* rather than content search:

- **Localized edits.** An aligner walks the old extent map and offers each new chunk the old chunk it overlaps most.
- **Rename-over.** Git, editors and package managers save by writing a temporary file and renaming it over the original, producing a new inode that an edit-based scheme never connects to the old file. The Workspace overlay records the replaced file when `rename()` runs and offers it as the base.
- **Recreate at a path.** A file deleted and recreated at the same path is offered the file the base held there.
- **Similarity fallback.** Whole files with no event-derived base consult a bounded, persisted resemblance index of eight-hash bottom-k signatures (8,192 entries, 704 KiB in total), in the spirit of super-features [31, 39].

Rename-over tracking reduced bytes stored for a replayed agent's Git index churn by 80% (1,463,988 to 297,095 bytes), and small-file base selection on rename-over and recreate cut payload by 44% on a new-path-per-version workload (§8.6).

6.5 Bounded chains

Deltas create read dependencies. Chains are bounded by depth (16 for whole files, 8 for chunks and metadata) and by work: a payload chain's decoded bytes must stay within $4\times$ the cutoff and its encoded bytes within $2\times$. When a candidate's chain reaches a limit, the dependent re-bases onto the chain's independent root rather than being stored in full. A base whose delta trials keep losing—for example the old last chunk offered to every appended chunk—is backed off after eight losses. There is no background repacking or garbage collection: a representation chosen at commit time is permanent, and the selection must therefore be cheap and bounded.

6.6 Metadata

Per-commit cost on small edits is dominated by metadata, not payload. Inode values are pooled, with leaves storing (serial, ordinal) pairs; values live in immutable, field-packed, checksummed blocks of at most 4,000 bytes. Inode table branches, directory pages and extent pages may be stored as COPY/INSERT programs against the page they replace, under the metadata depth cap.

6.7 Packs

New records are grouped by lane (tree pages, chunks, compact whole files, singletons), sealed into append-only packs of at most 256 KiB, and stored as database rows; triggers forbid

updating or deleting a sealed pack. A locator row maps each identifier to its pack, group and record.

7 Implementation and Performance Engineering

LayerFS is written in Rust, with a Node.js SDK and a command-line client. On Linux it serves Workspaces through FUSE using `fuser` 0.18 with three small patches (signed timestamps, explicit session lifecycle, and receive buffers sized to `max_write` rather than 16 MiB); on macOS it serves a loopback NFSv3 export. It negotiates 128 KiB reads and writes and keeps kernel write-back off. We optimized by instrumenting exact counters (SQL statements, Store round trips, overlay jobs, delta trials) and removing work, keeping one construction producer per commit.

Isolating Workspaces. The first daemon served all Workspaces through one overlay database owned by a single thread with class queues and admission credits; it saturated near 7,000 file operations per second. Giving each Workspace its own overlay file and a single short-hold lock cut statements by 12–61% across our case set and a four-Workspace commit from 67.4 s to 13.4 s (one sample; one dense-rewrite case regressed by 45%).

Flat commits on long histories. Commit cost first grew with Branch length, because each commit re-read every inode leaf under an edited branch page. Storing inode values as Store rows and reading only children a change reaches reduced pack reads per commit from 2,198 to 15 at 2,000 prior commits; mounted commit latency stayed at a 13–16 ms median from 0 to 950 earlier commits.

Reads never wait for writes. Removing an accidental dependency of reads on the publication lane cut p95 read latency in a multi-agent replay from 96 ms to 10 ms.

Large new data. Appending 500 MiB once took 6.9 s to commit. Seven fixes—a producer thread feeding the Save through a bounded queue, backing off losing delta bases (26,966 trials to 22), batched locator lookups (53,500 calls to 144), hashing each pack once, reading overlay rows in whole windows, moving WAL checkpoints off the commit path, and reclaiming after the reply—brought it to 1.40 s. An eighth trades 1% more storage for 37% faster log-append commits; we report both.

Import and exec. A 500 MiB import from macOS into the Linux worker fell from 32.1 s to 5.0 s after replacing base64 frame encoding with binary attachments and pipelining parts. Restoring file timestamps when unpacking Sandbox images stopped Python from recompiling every import, cutting a typical edit step from about 105 ms to 62 ms.

8 Evaluation

We start with a 13-hour build by nine agents that used LayerFS as their only filesystem, then ask: (1) how much does recording everything cost compared with agent and snapshot filesystems; (2) which storage mechanism does the work, and when; (3) how fast is the per-call lifecycle; (4) where is LayerFS slower; and (5) is it correct.

8.1 Setup and methodology

All measurements ran on an Apple M3 Max (14 cores, 36 GiB) under macOS, with LayerFS and baselines inside Docker Desktop's Linux VM (8 vCPUs, 7.75–8 GiB), using SQLite Stores, one construction worker, and binaries, bundles and images pinned by hash. The host was shared with other workloads (1-minute load 4–29). Storage figures are exact byte counts. Most timings are single samples and we treat them as diagnostics; the ovnfs comparison uses three samples per case and reports medians, with a verified cold boundary before every commit and read: each system's files are flushed and evicted with `posix_fadvise(DONTNEED)`, and a boundary counts only when `fincore` reports zero resident bytes (1,014 of 1,014 did). Daemon caches, VM metadata caches and the host cache beneath the VM disk were not controlled. Every Branch head was verified against an oracle. [Before submission: rerun on dedicated native Linux, same route into every system, $N \geq 5$ samples with intervals.]

Driving LayerFS. Every harness drives LayerFS only through the public command line (Appendix H), in the sequence below; times in §8 include each command's process launch and routing.

```
# once per case
layerfs store create s     # or: store connect s <url>
layerfs machine start m    # docker or a VM
layerfs store push s m     # Store next to the Sandboxes
layerfs project init s/p <folder>
layerfs sandbox create s/p box m
# once per shell call
layerfs workspace mount box <branch>
layerfs workspace exec <ws> '<command>'
layerfs workspace commit <ws>
layerfs workspace unmount <ws>
# measurement and teardown
layerfs store stats s; layerfs project stats s/p
layerfs sandbox close box; layerfs store remove s
```

Baselines. ovnfs [32] (commit `5d54cb0`), which packs, compresses and delta-encodes its store, run on ext4 (no reflinks). Btrfs with zstd, with and without offline deduplication (bees), from an earlier unpaired run. Git packs as an offline storage reference: Git is not a live filesystem and its commit scans the tree, so we compare only its packed size.

Workloads. (i) *History shapes*: a real 65.0 MB, 9,404-file repository with a seeded edit model (70% line edits, 10% new files, 5% deletes, 10% renames, 5% appends; five operations per commit) on one or several Branches. (ii) *Large files*:

Run: 9 agents, 13 h 22 min, 4,354 commits, 211 Branches				
Commit kind	Commits	New (median)	Stored (median)	Total stored
File edits	3,051	29 KB	**0.9 KB**	0.07 GB
Builds, tests	1,195	9.8 MB	1.3 MB	4.68 GB
Merges	67	188 KB	16 KB	0.002 GB

Table 3: Storage of the nine-agent build by commit kind (totals in Figure 7). Every commit records the whole project tree, about 9 GB including build folders. LayerFS at `c5e58735bb`, release build, SQLite Store.

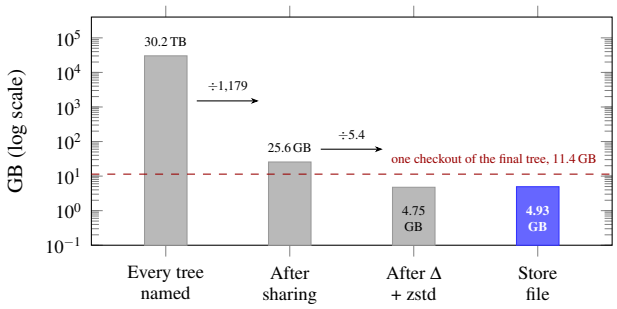


Figure 7: Where the nine-agent build's bytes went. 4,354 commits name 30.2 TB of whole trees; sharing removes all but 25.6 GB, deltas and compression leave 4.75 GB, and the Store file is smaller than one checkout of the final tree.

100 B overwrites, inserts and 16 KiB appends on 5–500 MiB incompressible files, and 500 MiB create, append, remove and truncate. (iii) *Real history*: 157 snapshots of a public agent-harness repository (every 100th of 15,632 commits), replayed by rewriting each changed file whole. (iv) *Single-mechanism micro cases*. (v) *Multi-agent runs* of an open-source coding harness on LayerFS: a 13-hour, nine-agent build on a 2.0 release build (§8.2) and two earlier runs on 1.x.

8.2 A 13-hour, nine-agent build

Nine agents—a lead and eight workers in four lanes—built a SQLite-compatible database engine in Rust through an open-source agent harness, with LayerFS as their only filesystem. Every shell call ran in a Workspace (mount, run, commit, unmount); each lane worked on its own Branch, and the lead merged lanes on an integration Branch and published to `main`. The run ended when a terminal check passed on a fresh fork of `main`: 15 crates and about 87,000 lines of Rust, 1,567 passing tests, and 16 of 16 acceptance checks against the reference `sqlite3`.

Storage. Every commit recorded the whole tree, about 9 GB including `target/`. The 4,354 commits on 211 Branches name 30.2 TB of trees, which reduce to 25.6 GB of distinct content and then to 4.75 GB after deltas and compression. The Store file is 4.93 GB, less than half of one checkout of the final tree (Figure 7). Edits are nearly free: the 3,051 file-edit commits stored a median 0.9 KB each, although every one recorded a 9 GB tree. Builds produced 98.5% of the stored bytes. Keeping ignored files and build outputs therefore costs what the agents *produce*, not what the tree *holds*.

Call	Median	p90	Calls
`workspace.mount`	7 ms	16 ms	1,121
`workspace.unmount`	14 ms	35 ms	1,121
`workspace.commit`	204 ms	1.06 s	1,186
`file.read`	6 ms	10 ms	4,926
`file.edit`	14 ms	25 ms	982
`branch.fork`	5 ms	8 ms	93
`branch.merge`	263 ms	479 ms	67
`branch.publish` (11 GB tree)	7 ms	11 ms	39

Table 4: LayerFS call latency in the nine-agent build, after the release build was installed mid-run.

Latency. Table 4 shows LayerFS calls over the run. Borrowing a Workspace cost a median 7 ms to mount and 14 ms to unmount across 1,121 shell calls. Forks took 5 ms, and publishing an 11 GB tree to `main` took 7 ms because a Layer names an existing root. Commits took a median 204 ms because many carried fresh build outputs.

State outlives compute. Midway through the run, a settings save in the host application reloaded the harness plugin and ended the live run. No work was lost: every agent's state was in its Branches, and the run resumed from them.

What went wrong. Of 59,331 LayerFS calls, 1.2% were refused, and 35 of 5,312 writes (0.66%) were refused because the Store was busy. Concurrent writes to the Store were sometimes refused with an operating system `NotFound` error, which agents worked around by retrying. Stopping an agent's turn left its command running in the Sandbox while Workspace status reported it idle. Copying a warm `target/` with a file call took about a minute. Two LayerFS changes were installed while the agents kept working: a release build replacing a debug build of the Linux bundle, and a fix that stopped unmount from waiting for a commit's deferred cleanup; the latencies above are from after both. The harness's operator checked the run every 10–15 minutes and steered it when it stalled.

8.3 Storage against agent filesystems

Table 5 compares LayerFS with ovnfs. On code-edit histories the imported base tree dominates the total, and ovnfs ends 7.6–10% larger. The per-commit cost differs far more: ovnfs adds 32–106 KB per commit and LayerFS 4.9–6.5 KB, a 6–22× gap. About two thirds of LayerFS's per-commit bytes are metadata. On large files the gap is structural: ovnfs stores each overwritten or inserted-into version in full (about 100% of raw bytes), while LayerFS stores the original plus 5–7 KB per commit (5–17% of raw); the 100 B overwrite itself costs

Case	Shape	Stored at end (MB)		Added per commit		Commit, cold (ms)	
		ovnfs	LayerFS	ovnfs	LayerFS	ovnfs	LayerFS
S1	1 Branch × 40 commits	26.37	**23.97**	32.2 KB	**5.1 KB**	83.8	**41.9**
S2	5 Branches × 3 commits	26.09	**23.88**	66.6 KB	**5.5 KB**	77.5	**35.8**
S3	fork chain 4 deep × 3	25.59	**23.81**	41.6 KB	**6.5 KB**	73.7	**39.1**
S4	5 Branches, same commit	25.62	**23.78**	105.7 KB	**4.9 KB**	70.6	**46.0**
L100	100 MiB, 100 B overwrite ×10	1,154.7	**106.3**	104.9 MB	**5.3 KB**	221.1	**24.5**
L10	10 MiB, 100 B overwrite ×20	221.5	**10.9**	10.5 MB	**5.5 KB**	190.9	**24.4**
I10	10 MiB, 100 B insert ×10	116.6	**10.9**	10.5 MB	**5.3 KB**	197.9	**25.5**
A5	5 MiB, 16 KiB append ×20	18.7	**5.9**	613 KB	**18.4 KB**	145.2	**19.5**
L500	500 MiB, 100 B overwrite ×5	3,146.9	**530.4**	524.3 MB	**7.4 KB**	603.9	**28.1**

Table 5: LayerFS versus ovnfs on code-edit histories (S1–S4, from a 65 MB, 9,404-file tree) and large-file edits. Medians of three samples; commit times are measured after a verified cold boundary and include each system's route (CLI-to-worker for LayerFS, `docker exec` for ovnfs). LayerFS at `b9c4adcb`; ovnfs at `5d54cb0`.

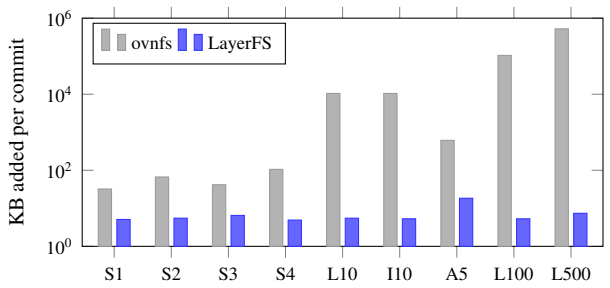


Figure 8: Bytes added per commit (log scale), from Table 5.

153 B of payload at every file size, and the rest is metadata (Figure 17 contrasts whole-file copy-up, block copy-on-write and LayerFS for this edit). Appends are the one large-file case where ovnfs compresses well (16% of raw); LayerFS stores the incompressible appended bytes nearly as is and ends at 5%.

8.4 Recording everything on real histories

157 snapshots of a real repository. Replaying 157 snapshots of a public agent-harness repository (4.94 GB of trees; 75,929 distinct files totalling 892 MB; two thirds of distinct files under 8 KiB), LayerFS keeps every snapshot as a mountable commit in 100.1 MB, 49× less than the trees. Btrfs with zstd snapshots required 1,994.6 MB (19.9×), and adding the bees deduplicator raised it to 2,068.9 MB including its state, because sharing metadata grew by more than the data it merged. A Git pack of the same states is 55.1 MB, smaller than LayerFS. The replay rewrites each changed file whole, which removes Btrfs's in-place extent sharing by construction, and the Btrfs run was not paired with this LayerFS build; we rerun it in the final evaluation. [Release-labelled v1.0.0 run at `e5c7aaae05`; rerun on 2.0.]

History shapes. Table 6 breaks down the reduction. On the real history at two sampling strides (D10, D03), exact deduplication contributes 1.5–2.8× and delta encoding plus

Case	Raw MB	Stored MB	Raw/ stored	Dedup	Δ+zstd
D10 17 states	561	47.9	11.7×	1.48×	9.26×
D03 53 states	1,677	61.0	27.5×	2.84×	11.6×
H01 500 commits	32,477	26.5	1,226×	–	–
H03 50 same Br.	3,316	23.8	139×	–	–
H05 20 proj./1 Store	7,802	28.3	276×	–	–
H06 20 proj./20 Stores	7,802	471.4	16.6×	–	–

Table 6: Reduction on real and seeded histories (SQLite, `7691ac20`). *Dedup* is raw over new canonical bytes; *delta+zstd* is new canonical over new stored bytes. H rows start from a 65 MB imported base.

compression 9.3–11.6×; Store overhead (database pages, indexes and free space around the packs) adds 17–20%. Against the offline Git reference, LayerFS is 1.15–1.27× larger. On seeded histories over a 65 MB base, 500 commits add about 5.9 KB each (2,990 B of new packs, two thirds metadata); fifty identical Branches add about 270 KB in total; and twenty projects sharing one Store need 16.7× less space than twenty separate Stores.

Earlier multi-agent runs. Two earlier runs of the same harness, on a 1.x build, used LayerFS as the agents' only filesystem. Building a trip-planning web application (1 lead and 7 worker agents, 56 minutes) produced 147 commits on 31 Branches, whose snapshots total 91.5 MB, in a 1.21 MB Store. Building a SQL database in Rust with every build's `target/` directory committed (8 agents, 4 hours 16 minutes) produced 913 commits on 56 Branches whose snapshots total 4.93 TB, in an 8.90 GB Store—less than one copy of the final tree (13.5 GB). [Site-reported v1 runs; cite the run record.]

8.5 Commit granularity decides what saves space

Figure 9 splits the reduction into the two stages of §6. When agents commit after every tool call, consecutive states differ

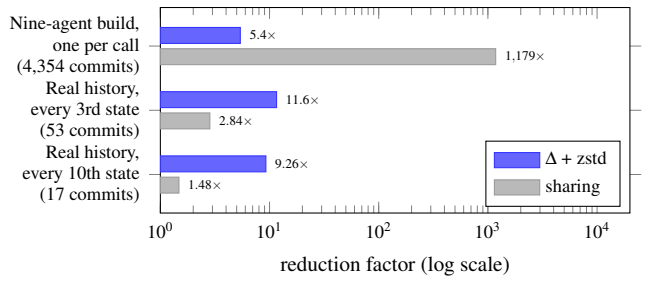


Figure 9: Which mechanism does the work. *Sharing* is logical bytes over distinct canonical bytes; *delta + zstd* is distinct canonical bytes over stored bytes. Per-call commits make sharing dominant; coarse snapshots shift the load to deltas.

Micro case	Raw/stored	Dedup	Delta+zstd
M01 10,000 tiny files	0.91×	0.94×	1.05×
M02 tree + 10 copies	16.97×	11.90×	1.45×
M03 1% in-place edits	29.27×	1.00×	32.12×
M04 same, rename-over	28.02×	1.00×	32.04×
M05 same, new path	25.15×	1.00×	30.07×
M06 64 MiB random + 5 edits	5.87×	5.90×	1.00×
M07 random / zeros / text	4.29×	2.99×	1.45×
M08 SQLite DB, 1% churn	13.73×	3.21×	4.44×

Table 7: Single-mechanism cases (SQLite Store, `7691ac20`).

in a few files, so nearly every object in the next commit already exists: sharing reduces 30.2 TB of trees to 25.6 GB, a 1,179× reduction, and deltas plus compression contribute a further 5.4×, mostly on fresh build outputs. When the same kind of source tree is snapshotted coarsely—every 3rd or 10th of 157 historical states, with each changed file rewritten whole—many files change between snapshots, each a new version close to an old one. Sharing then contributes only 1.5–2.8× and delta encoding carries the load (9.3–11.6×). The per-call model that makes Workspaces ephemeral is also what makes sharing so effective; delta encoding matters for rewritten files, build outputs and coarse imports. The two workloads differ in more than granularity—the build includes compiled outputs, the replay does not—so we read the figure as indicative. [Control for content: replay the nine-agent run's states at strides 1, 3 and 10.]

8.6 Mechanism contributions

Table 7 isolates mechanisms. Copies (M02) are absorbed by exact reuse; small edits to text files (M03) by deltas, whether the editor writes in place, renames over the original (M04) or writes a new path (M05, which falls back to the similarity index); edits to a large random file (M06) by chunk reuse, with each edit adding one or two chunks; and a SQLite database under row churn (M08) by both. Tiny files (M01) are the one case where the Store exceeds raw bytes, by about 10%, because per-object metadata outweighs the content.

Operation (Docker, SQLite Store)	Time
Mount a Branch as a Workspace	18.8 (13–32) ms
Fork + mount	31–41 ms
Run a no-op command (route included)	22–42 ms
Commit, no change (`UpToDate`)	11–15 ms
Commit, small edit (warm / cold)	20–33 / 36–46 ms
Commit, 100 B edit in a 500 MiB file	15–28 ms
Unmount	16.9 (12–50) ms
Sandbox create, warm VM†	0.39 (0.29–0.90) s
Sandbox create, first on a fresh VM	3.4–5.3 s

Table 8: Per-call lifecycle costs through the public CLI, including client launch and routing. Mount, unmount and warm create are medians (ranges) over 32 test rows of one campaign; other rows are single samples, except cold commits (median of 3). Mount takes 17–18 ms in Lima and Apple container VMs as well. †Earlier per-container Sandbox route; warm create on the current namespaces route is not yet measured.

Event-sourced base selection matters most where content search fails. Before rename-over tracking, a replayed agent's Git index—rewritten through `index.lock` on every Git command—received no delta base; with it, stored bytes for that workload fell from 1,463,988 to 297,095 (§6.4). A storage round that changed only base selection, metadata programs and layout (with the codec frozen) cut net Store bytes by 27–45% on small-file edit sequences (comparing closed Stores), by 3–18% on SQLite history workloads with payload unchanged, and per-commit metadata by 19–41%. [Add controlled ablations: CDC off, deltas off, similarity index off, rename predecessor off, metadata deltas off; cutoff and depth sweeps.]

8.7 Per-call latency

Table 8 shows the lifecycle of one shell call. Mount, an unchanged commit and unmount each take a median of 11–19 ms, so the per-call overhead of borrowing a Workspace is tens of milliseconds—small next to a model call. Creating a Sandbox is three orders of magnitude slower, which is why Sandboxes are pooled and only Workspaces are per call. Under real agent load the per-call costs were lower still: a median 7 ms mount and 14 ms unmount across 1,121 shell calls (Table 4). Commit latency is also flat in history length: the median mounted commit stayed at 13–16 ms from 0 to 950 earlier commits on one Branch (§7). Against ovnfs, LayerFS ran the edit step of each code-history commit 1.3–1.7× faster and committed 1.5–2.2× faster cold (Table 5); on large files it ran the edit 1.5–4.9× faster and committed 7.4–21× faster, because ovnfs copies the whole file up on first write and rehashes it at snapshot. [Missing: mount and commit latency vs tree size (10^3–10^6 files) and vs fork depth; time to first passing test vs clone + install.]

Case (cold, median of 3)	ovnfs	LayerFS
Read 9,404-file tree (65 MB)	**3.53 s**	5.31 s
Same tree after 40 commits	**3.96 s**	5.32 s
Read one 500 MiB file	**0.56 s**	3.33 s
Same file after 5 commits	**0.58 s**	3.31 s
Write new 500 MiB file / commit	**1.76 / 1.00 s**	2.67 / 1.36 s
Truncate 500 MiB / commit	0.97 / 0.59 s	**0.05 / 0.05 s**

Table 9: Cold reads and bulk data. History depth barely affects LayerFS reads (under 1%).

8.8 Where LayerFS is slower

Table 9 shows the costs of LayerFS's design. **Reads**: a cold read of a 500 MiB file reaches 158 MB/s against ovnfs's 943 MB/s, and warm reads are similarly slow, which points at the request path rather than storage: LayerFS negotiates 128 KiB reads with one outstanding background request, so read-ahead is serialized through the daemon. History depth is not the cause—reading after five overwrite commits differs by under 1%. Cold whole-tree reads are 1.3–1.5× slower because each file's content is a separate Store read with no directory-level prefetch. **New data** is written twice: once into the overlay at `write(2)` time (write-back is off so capture is exact) and again into the Store at commit. **Import** of a 65 MB tree takes 2.2–3.1 s, 6–8× slower than ovnfs, partly because LayerFS's route crosses the Docker Desktop port forward while ovnfs imports from inside its container; import is a one-time cost per project. [Profile and fix the read path before submission; this is the weakness reviewers will press.]

Against JuiceFS with PostgreSQL metadata on the same VM (JuiceFS's own `bench`, four workers), LayerFS wrote small files 8.8×, read them 3.6× and stat'ed them 2.6× faster, but streamed 1 GiB files at 0.26–0.30× JuiceFS's throughput. The comparison is not durability-matched: JuiceFS ran with `fsync` and synchronous commit on, LayerFS with its disposable profile.

8.9 Correctness

The format deliberately omits set-id bits, extended attributes, FIFOs, sockets and device files; within that scope we ran three suites. fsx ran 3.06 million operations across seven variants (256 KiB–64 MiB files, including `mmap` and `O_DIRECT`) with zero miscompares on a 1.0 build, and its current-build subset passes in all nine platform and backend configurations. Four fsstress workers performed 40,000 concurrent operations; committing, forking and remounting the result reproduced all 3,755 entries exactly. On the current build, fsstress passes 8 of 8 configurations; an intermediate build lacking support for hard links to symbolic links failed 7 of 9. A pjdfstest subset (29 assertions over `open`, `rename`, `chmod` and `symlink`) passes in all nine configurations. [Run full pjdfstest and an xfstests generic subset; independent committed-content verification was skipped in the six-suite campaign.]

9 Discussion and Limitations

What "durable" means here. History is durable in that it outlives every Workspace, Sandbox and agent: losing any of them loses no committed work, as the nine-agent run showed when its live session ended mid-run. It is not crash-durable in the `fsync` sense. The SQLite Store runs WAL with `synchronous=OFF` and Workspace overlays are never synced, so committed history survives process crashes but not kernel or VM crashes. For agent workloads, where the costliest loss is the agent's time and a recent commit can be recomputed, we chose latency; a crash-durable profile exists but was not evaluated.

No garbage collection. The Store only grows; deleting a Branch does not reclaim its objects, and representations chosen at commit time are never improved by repacking. Both are future work.

Single-writer metadata. Object construction proceeds in parallel, but publications on one Store are serialized through the lane, and on SQLite each pack batch also takes SQLite's write lock and the Store directory lock briefly. With nine agents, 0.66% of writes were refused as busy after a bounded wait; we have not yet measured contention systematically as the number of agents grows.

Search at scale. Section 5.3 argues from the design. Wide searches would stress three limits. Every Branch on a Store shares one publication lane, so thousands of concurrent rollouts that commit every step put the lane in the contended regime it was not sized for (§5.4). A Sandbox hosts a bounded number of Workspaces, so wide fan-out needs a pool of Sandboxes. Without garbage collection, dropped rollouts stay stored, each costing only its changed bytes. A node also restores files, not processes: services a rollout started must be started again. [Experiment: synthetic MCTS driver, branching 8, depth 4 (about 4,700 nodes): fork, mount and commit latency against depth and fan-out; bytes per node; lane wait and Busy refusals against concurrency; time to first test from a deep node against clone + install.]

Semantics. Commit captures the filesystem, not processes, memory, sockets or output; unflushed `mmap` pages and still-running descendants are not fenced. Because the filesystem does not supervise commands, stopping an agent's turn can leave its command running while Workspace status reports idle; the Sandbox runtime, not the filesystem, must cancel it. On macOS, the NFS route cannot host SQLite in WAL mode inside a Workspace.

Open issues. The nine-agent run exposed concurrent writes refused with an operating system `NotFound` error on the Docker Store, and slow file-call copies of large build directories (about a minute for a warm `target/`). Both are open.

Premise data. Our agent-time analysis covers one developer's runs and classifies shell commands heuristically. A cleaner study would record, per tool call, monotonic start and end times and a harness-assigned intent class; per model call, request, first-token and last-token times; per shell call, whether it touched only Workspace files (from a filesystem trace); and task identifiers across many users and codebases.

Evaluation scope. All numbers come from one Apple-silicon host running Docker Desktop, mostly single samples, with different routes into LayerFS and ovnfs; the nine-agent build is one run, steered by an operator. Several key claims—mount cost independent of tree size, fork cost independent of depth, time to first passing test without reinstalling, and rollout throughput—follow from the design but still need direct measurement.

10 Related Work

Deduplication and delta compression. LBFS [21] introduced content-defined chunking for file systems; Venti [27] and Data Domain [40] built content-addressed archival and backup stores; FastCDC [36] is the chunker family LayerFS uses. Resemblance detection with super-features [31, 39] selects delta bases by content; LayerFS prefers bases derived from filesystem events and uses resemblance only as a fallback. Git [2] delta-compresses offline in `gc`; LayerFS decides once, inline, under bounded work.

Versioning file systems. Elephant [30], ext3cow [25], WAFL [14] and Btrfs [29] keep snapshots of a live file system. They share blocks that stay in place; LayerFS additionally shares content across rewritten files, paths, Branches and projects, and makes every tool call a named, forkable state.

Containers and serverless. Overlay and layered container filesystems [4, 13] make container start-up fast but copy whole files on write. Serverless platforms [16, 22] and snapshot-restore systems [5, 12, 34] make compute ephemeral for stateless functions; LayerFS applies the same separation to stateful agents by moving the state into a versioned store. Image-based sandbox filesystems have moved from FUSE over overlayfs to lazily served block devices with block-level copy-on-write, as in Modal's two generations [7, 19]; Appendix G compares them with LayerFS.

Database-backed file systems. Inversion [24], TableFS [28] and JuiceFS [3] keep file system state in databases. AgentFS [33] stores an agent's files in SQLite without history. LayerFS uses a database as the single home of an immutable, versioned object graph.

Agent infrastructure. Agent benchmarks and frameworks [15, 37, 35] run each task in its own container. Sampling and search methods [9, 38] multiply the number of trajectories per task; LayerFS makes each trajectory a Branch.

11 Conclusion

LayerFS is a recursive filesystem for ephemeral computing: it gives agents ephemeral Workspaces over durable shared history. File calls need no computer; shell calls borrow a complete Workspace—dependencies, build outputs and ignored files included—for milliseconds, commit, and go away; and every step becomes a commit that any agent can fork, at any fork depth, without copying data. Recording everything is affordable because commits share structure first and compress second: in a nine-agent build, 4,354 whole-tree commits fit in less than half of one checkout. Because agents commit at every tool call, sharing does most of the work.

References

[1] E2B: Open-source cloud sandboxes for AI agents. `https://e2b.dev`, 2025.

[2] Git. `https://git-scm.com`, 2025.

[3] JuiceFS: A distributed POSIX file system built on Redis and S3. `https://juicefs.com`, 2025.

[4] Overlay filesystem. Linux kernel documentation, `https://docs.kernel.org/filesystems/overlayfs.html`, 2025.

[5] Alexandru Agache, Marc Brooker, Andreea Florescu, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. Firecracker: Lightweight virtualization for serverless applications. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020.

[6] Michael Armbrust, Tathagata Das, Liwen Sun, Burak Yavuz, Shixiong Zhu, Mukul Murthy, Joseph Torres, Herman van Hövell, Adrian Ionescu, Alicja Łuszczak, Michał Świtakowski, Michał Szafrański, Xiao Li, Takuya Ueshin, Mostafa Mokhtar, Peter Boncz, Ali Ghodsi, Sameer Paranjpye, Pieter Senster, Reynold Xin, and Matei Zaharia. Delta Lake: High-performance ACID table storage over cloud object stores. *Proceedings of the VLDB Endowment*, 13(12), 2020.

[7] Jonathon Belotti. Fast, lazy container loading in Modal.com. Modal blog, `https://modal.com/blog/jono-containers-talk`, 2024.

[8] Andrei Z. Broder. On the resemblance and containment of documents. In *Compression and Complexity of Sequences*, 1997.

[9] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. arXiv:2407.21787, 2024.

[10] Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. A survey of Monte Carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1), 2012.

[11] Yann Collet and Murray Kucherawy. Zstandard compression and the application/zstd media type. RFC 8878, 2021.

[12] Dong Du, Tianyi Yu, Yubin Xia, Binyu Zang, Guanglu Yan, Chenggang Qin, Qixuan Wu, and Haibo Chen. Catalyzer: Sub-millisecond startup for serverless computing with initialization-less booting. In *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020.

[13] Tyler Harter, Brandon Salmon, Rose Liu, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Slacker: Fast distribution with lazy Docker containers. In *USENIX Conference on File and Storage Technologies (FAST)*, 2016.

[14] Dave Hitz, James Lau, and Michael Malcolm. File system design for an NFS file server appliance. In *USENIX Winter Technical Conference*, 1994.

[15] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[16] Eric Jonas et al. Cloud programming simplified: A Berkeley view on serverless computing. Technical Report UCB/EECS-2019-3, University of California, Berkeley, 2019.

[17] Levente Kocsis and Csaba Szepesvári. Bandit based Monte-Carlo planning. In *European Conference on Machine Learning (ECML)*, 2006.

[18] Yiren Lu. How Modal speeds up container launches in the cloud. Modal blog, `https://modal.com/blog/speeding-up-container-launches`, 2024.

[19] Nitin Maddi and Amit Prasad. Building fast filesystems for VMs. Modal blog, `https://modal.com/blog/vm-sandbox-filesystem`, 2026. October 16, 2026.

[20] Modal. Sandbox snapshots. Modal documentation, `https://modal.com/docs/guide/sandbox-snapshots`, 2026.

[21] Athicha Muthitacharoen, Benjie Chen, and David Mazières. A low-bandwidth network file system. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2001.

[22] Edward Oakes, Leon Yang, Dennis Zhou, Kevin Houck, Tyler Harter, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. SOCK: Rapid task provisioning with serverless-optimized containers. In *USENIX Annual Technical Conference (ATC)*, 2018.

[23] Jack O'Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O'Hearn. BLAKE3: One function, fast everywhere. `https://github.com/BLAKE3-team/BLAKE3-specs`, 2020.

[24] Michael A. Olson. The design and implementation of the Inversion file system. In *USENIX Winter Technical Conference*, 1993.

[25] Zachary Peterson and Randal Burns. Ext3cow: A time-shifting file system for regulatory compliance. *ACM Transactions on Storage*, 1(2), 2005.

[26] Amit Prasad. VM sandboxes: Full computers for agents. Modal blog, `https://modal.com/blog/vm-sandboxes-agent-computers`, 2026.

[27] Sean Quinlan and Sean Dorward. Venti: A new approach to archival storage. In *USENIX Conference on File and Storage Technologies (FAST)*, 2002.

[28] Kai Ren and Garth Gibson. TABLEFS: Enhancing metadata efficiency in the local file system. In *USENIX Annual Technical Conference (ATC)*, 2013.

[29] Ohad Rodeh, Josef Bacik, and Chris Mason. BTRFS: The Linux B-tree filesystem. *ACM Transactions on Storage*, 9(3), 2013.

[30] Douglas S. Santry, Michael J. Feeley, Norman C. Hutchinson, Alistair C. Veitch, Ross W. Carton, and Jacob Ofir. Deciding when to forget in the Elephant file system. In *ACM Symposium on Operating Systems Principles (SOSP)*, 1999.

[31] Philip Shilane, Mark Huang, Grant Wallace, and Windsor Hsu. WAN optimized replication of backup datasets using stream-informed delta compression. In *USENIX Conference on File and Storage Technologies (FAST)*, 2012.

[32] Stencil. ovnfs. `https://github.com/stencil-hq/ovnfs`, 2026.

[33] Turso. AgentFS: A filesystem for agents. `https://github.com/tursodatabase/agentfs`, 2025.

[34] Dmitrii Ustiugov, Plamen Petrov, Marios Kogias, Edouard Bugnion, and Boris Grot. Benchmarking, analysis, and optimization of serverless function snapshots. In *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2021.

[35] Xingyao Wang et al. OpenHands: An open platform for AI software developers as generalist agents. In *International Conference on Learning Representations (ICLR)*, 2025.

[36] Wen Xia, Yukun Zhou, Hong Jiang, Dan Feng, Yu Hua, Yuchong Hu, Qing Liu, and Yucheng Zhang. FastCDC: a fast and efficient content-defined chunking approach for data deduplication. In *USENIX Annual Technical Conference (ATC)*, 2016.

[37] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[38] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[39] Yucheng Zhang, Wen Xia, Dan Feng, Hong Jiang, Yu Hua, and Qiang Wang. Finesse: Fine-grained feature locality based fast resemblance detection for post-deduplication delta compression. In *USENIX Conference on File and Storage Technologies (FAST)*, 2019.

[40] Benjamin Zhu, Kai Li, and Hugo Patterson. Avoiding the disk bottleneck in the Data Domain deduplication file system. In *USENIX Conference on File and Storage Technologies (FAST)*, 2008.

Appendices

These appendices describe LayerFS 2.0 (branch `layerfs_v2_0_0`, commit `8cfb6e2734`) at the level of data structures, algorithms and architecture. Constants are the source's defaults. Pseudocode follows the source's control flow and omits error paths, counters and cache plumbing.

A System Architecture and Concurrency

Layers and crates. LayerFS 2.0 is 26 Rust crates in four layers, `core` < `engine` < `control` < `api`; a layer depends only on layers below it, and a guard script enforces this. Core holds environment-free algorithms. Engine turns them into a persistent Store and mounted Workspaces. Control knows which machines, Stores and Sandboxes exist and where each call runs. The API layer is one call table (73 calls in 10 families) behind the CLI, Node and Python front ends (Table 10).

Layer	Crate (`layerfs-` omitted)	Responsibility
core	content	Objects and identities, CDC files, trees, diff, three-way merge, `.lww`
core	storage	Delta and zstd encoding, packs, Saves, authenticated reads
core	history	LayerStacks, Branches, Commits, Layers
core	telemetry	Timing trees
engine	persistence	The global Store (SQLite or PostgreSQL), admission, writer lock, publication lane
engine	store-service	In-process Store calls, Commit publication, private Workspaces
engine	project	Project initialization (directory → first Layer)
engine	overlay	Per-Workspace disposable SQLite overlay
engine	workspace	Workspace semantics, shared object cache, Commit construction
engine	mount-ports, fuse, nfs	Kernel routes: shared ports, FUSE (Linux), loopback NFSv3 (macOS)
engine	bridge, daemon	Authenticated channel (Noise KK); the Sandbox daemon
control	home, machine, store, sandbox, control	Records and locks; machines and workers; Stores; Sandboxes and exec
control	platform, sandbox-backend, bucket	Drivers: platform adapters; namespaces or host-process Sandboxes; backups
api	api, cli, node, python	Call table and front ends

Table 10: Crates at `8cfb6e2734`.

Processes. The agent's SDK process runs every Store call—file reads and writes, fork, merge, publish—in-process, opening the Store directly. A file write commits through a private, never-mounted Workspace (up to 64 per process) using the same Commit pipeline as a mount. A *Sandbox* is one daemon on a machine's kernel plus the commands run beside it. The daemon opens the same Store, holds up to 16 Workspaces and serves each through FUSE (Linux) or a loopback NFSv3 export (macOS). Each Workspace has its own SQLite overlay file, lock and kernel mount. All Workspaces of a daemon share one 64 MiB object cache keyed by object identifier, so Workspaces on different Branches reuse identical objects with no invalidation protocol. Commands run through the Sandbox runtime (namespaces and a cgroup on Linux, a host process on macOS); the daemon only serves files. When a SQLite Store lives on a kernel the caller cannot share (a Docker, Lima or Apple-container VM, or Windows), calls go once to that machine's *worker*, which listens on loopback, authenticates a secret, accepts one request per connection and never retries.

Parallel objects, single-writer metadata. Writes are ordered at two levels: within a process by a FIFO admission queue (waiting is readiness, never a retry), and across processes by an exclusive `flock` on the Store's directory followed by `BEGIN IMMEDIATE` (SQLite), or by `SELECT ... FOR UPDATE` on a one-row `store_lane` table (PostgreSQL). Coordinated writers queue rather than fail; on PostgreSQL the wait is bounded by `lock_timeout` (30 s), after which the call fails `Busy`. Reads never take either lock: packs, locators and value blocks are immutable once visible.

A Commit splits its work around a *publication lane*, the writer lock plus an admission turn held by one thread across several short transactions. Everything that grows with the Commit runs before the lane: capture, construction on one producer thread, encoding, compression and pack sealing, a full three-way merge when the head has moved, and validation. The Save publishes packs as it goes, in batches of at most 4 MiB (SQLite) or 16 MiB (PostgreSQL), each an ordinary short write. On PostgreSQL the pack bodies are copied in *before* the lane row is locked, so only locator inserts wait on it; on SQLite the CPU work runs in parallel across Commits while the batch writes themselves serialize on the directory lock. Inside the lane the Commit re-reads the head, merges only what landed since its pre-lane read (at most 1,024 inodes, else `LaneMerge`) and advances the Branch in one transaction that checks the expected head. Figure 10 shows two concurrent Commits.

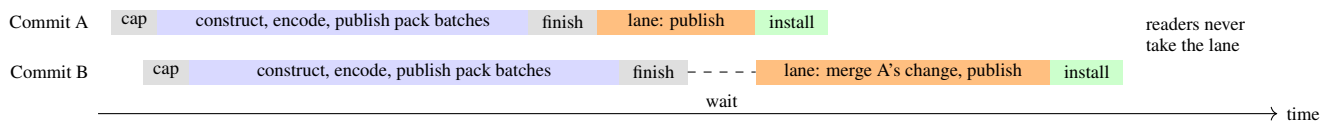


Figure 10: Two concurrent Commits on one Branch. Construction, compression and pack batches overlap; only the lane serializes, and B merges only the change A published.

Read path. A read that misses the kernel page cache (1) arrives as a FUSE request of at most 128 KiB, with one background request outstanding; (2) runs one overlay job under the Workspace lock that decides locally written bytes; (3) reads inherited bytes *outside* that lock from the shared object cache; (4) on a miss, issues one batched Store demand, plus 8 MiB of read-ahead for sequential streams (at most 16 MiB held per reader); and (5) decodes delta bases and zstd frames through a 2 MiB pack cache, re-hashing every object against its identifier. Mount reads only the root object. Per-request round trips and decoding, not history depth, dominate read cost.

Lifecycles and custody. A Sandbox lives from creation until it is explicitly closed or its machine stops. A Workspace is mounted, serves any number of processes and Commits, and is unmounted; unmount refuses uncommitted changes unless told to discard them. A Commit ends `Committed` (possibly after merging a moved head), `UpToDate`, `Conflict` (marker blocks become the Workspace's local base and the next Commit is refused while complete markers remain), or uncertain. Every hop makes one attempt; a lost reply is reported as unknown and its custody (published outcome, captured reader, operation intent) is retained until inspected. Committed history survives a daemon or Sandbox loss, which loses only the disposable overlays. Under the only permitted SQLite profile (WAL, `synchronous=OFF`) the Store survives process termination, but a kernel or VM crash can lose or damage it.

B Committed State: Objects, Trees and Packs

All committed state of a Store lives in one SQLite file or PostgreSQL schema as two kinds of data: immutable, content-addressed *objects* that encode filesystem trees, and a small set of mutable *history* rows that name tree roots.

Objects. Every object is a canonical byte string in a fixed envelope (Table 11). Its identifier is BLAKE3(`"layerfs/object/v2\0"` ‖ canonical bytes), 32 bytes. The value begins with an 8-byte role magic, so equal payload bytes under different roles have different identifiers. Thirteen roles exist: whole file, chunk, extent leaf and branch, file state, inode leaf and branch, directory leaf and branch, filesystem root, attribute leaf and branch, and symlink. Identity-bearing integers are big-endian. Decoders refuse padding, trailing bytes, nonzero flags, unknown versions and unsorted rows rather than repairing them, so each logical value has exactly one encoding, and references are always derived from the bytes, never supplied by a caller.

Structure	Layout (bytes)
Envelope (13)	`"LFSO"` [0:4] · kind `0x01` [4] · payload_len u32 [5:9] = value_len+4 · value_len u32 [9:13] · value
Tree-page header (31)	magic [0:8] · version u16 · role · level · flags= 0 · count u16 · two subtree totals u64
Whole file	`"LFS5SML\0"` · version u16 · payload (canonical = raw + 23)
Chunk	`"LFS4CHK\0"` · payload ≤ 32,768
File state (93)	`"LFS4MAP\0"` · version · length u64 · extents u64 · level · profile id (32) · mapping root (32)
Extent leaf entry (40)	chunk id (32) · offset into chunk u32 · length u32
Extent branch entry (48)	cumulative end bytes u64 · cumulative end extents u64 · child id (32)
Inode value (73)	kind (bit 7 = inline) · link count u64 · content root (32) · {attribute root (32) | mode u32 · mtime s i64 · ns u32 · 16 zero}
Filesystem root (116)	`"LFS6FSR\0"` · version · role · profile id (32) · scope id (32) · root serial u64 · inode-table root (32)

Table 11: Canonical byte layouts.

Trees. A filesystem root names an inode table: a B^+-tree keyed by 64-bit serial whose leaves hold 50–100 rows of (serial, 73-byte inode value) and whose branches hold 64–127 (maximum serial, child) rows. A directory's content root is its own B^+-tree from raw-byte name to serial; a name binds a serial, never an object, so a hard link is two names for one serial. Tree pages are at most 8,192 bytes, and directory and attribute pages are at least 40% full except at the root. A regular file below the

cutoff (128 KiB by default) is one whole-file object; a larger file is a file state over an extent B^+-tree of 64–128 entries per page whose leaves map logical ranges to *slices*—(chunk, offset, length)—of content-defined chunks (Figure 11). Slices let a localized edit keep the untouched parts of boundary chunks, and cumulative totals in branch entries locate an offset with one binary search per level. Holes are not special: a run of zeros is a sequence of identical all-zero 32 KiB chunks whose identical mapping pages are shared by identity. Profile identities hashed from the frozen chunking and page parameters are embedded in each file state and root and checked on decode. Updates rewrite only root-to-change paths; every unread subtree keeps its identity.

```
FilesystemRoot --> InodeLeaf (root page of the inode table)
                    1: Dir   --> DirectoryLeaf { "a.bin" -> 2, "b.txt" -> 3 }
                    2: File  --> FileState(len, extents, level 1, profile)
                    3: File  --> WholeFile "LFS5SML\0" | bytes
FileState --> ExtentBranch [(end0,n0,L0) (end1,n1,L1)]
                L0 ExtentLeaf [(C1,0,len1) (C2,0,len2) ...]      (64-128 entries)
                L1 ExtentLeaf [(C2,4096,len) (C9,0,len9) ...]    <- a slice of C2
                     |
                Chunk C1, C2, C9 ...   (shared across files, versions, Branches, projects)
```

Figure 11: One directory and one chunked file.

Physical records. Identity excludes physical form. Payload objects (whole files and chunks) are stored as `FULL` (a zstd frame), `STORED` (raw bytes, chosen by an incompressibility probe) or `PREFIX` (a zstd frame compressed against one base object's payload, recording the 32-byte base identifier). Inode branches and directory and extent pages may be stored as a COPY/INSERT program against a predecessor page (`COPY` $= 0 \cdot$ offset u32 $\cdot$ length u32; `INSERT` $= 1 \cdot$ length u32 $\cdot$ bytes), kept only when strictly smaller. Inode leaves are pooled: each 73-byte value is replaced by a 4-byte ordinal, and values live in append-only, field-packed blocks of at most 4,000 bytes and 255 entries ending in a CRC32C. Every read rebuilds canonical bytes, base first along a delta chain, and hashes each element against its identifier.

Packs. Records are grouped (at most 8,191 records and 64 KiB per group) into immutable packs of at most 256 KiB in four lanes: ordinary (tree pages, optionally zstd-compressed per group), chunk, whole file, and singleton for one record too large for a pack. A pack is a 24-byte header (`"LFPACK"`, version, group count, used length), a directory of group offsets, encoded and decoded lengths and codecs, and the group bodies; its key is the SHA-256 of its sealed bytes. A locator row maps each object identifier to (role, canonical length, pack, group, record).

Group	Table	Key → contents	Written by
Physical	`pack`	pack id → SHA-256, length, body (update and delete refused by trigger)	object batches
	`object_location`	object id → role, canonical length, pack, group, record	object batches (insert if absent)
	`inode_value_block`	first ordinal → ≤255 packed values + CRC32C	object batches
	`content_signature`	ring slot → object id, 32-byte sketch	object batches (upsert)
	`store_policy`, `store_totals`	cutoff, depth caps, next pack id and ordinal; byte totals	reservation, publication
History	`layer_stack`	id → name, scope, profile, head Layer	init, `add_layer`
	`layer`	id → stack, parent, root, source Branch and Commit	`add_layer`
	`commit`	id → root, parent(s), base Layer, message, statistics, `.lww` decisions	publication lane
	`branch`	id → name, base Layer, forked-from node, head Commit	fork, merge, lane
	`scope_allocator`	scope → serial high-water mark	serial reservation
Lineage	`layer_lineage`, `commit_lineage`, …	heights, first-parent runs, absorption	with each Layer or Commit
Hints	`binding_hint_block`, `file_digest`	name-binding hints; file-bytes digest → root	before the lane

Table 12: Store tables (SQLite; PostgreSQL is equivalent). Object batches run outside the publication lane; only Commit insertion and head moves run inside it.

History and invariants. History identifiers are domain-separated BLAKE3 digests with a one-byte kind tag; a Commit identifier covers its root, parents, base Layer and message but not its timestamp. Lineage tables record each Commit's first-parent run and depth, so ancestry queries walk run segments rather than individual Commits. The Store maintains six invariants: (i) every read authenticates; (ii) one encoding per value; (iii) references are derived from bytes; (iv) packs, locators and value blocks are append-only; (v) pack identifiers and inode serials are never reused; (vi) Commits and Layers are immutable, and only Branch and stack heads move, by expected-head checks. (Mutable rows besides heads are counters, the similarity ring and the file-digest hint table.)

C Workspace Overlay and Request Path

Overlay database. Each mounted Workspace owns one SQLite file created by a byte copy of a schema-only template built at daemon start; mount therefore costs a fixed-size file copy and one root-object read, independent of the tree. The file runs a disposable profile (`journal_mode=MEMORY`, `synchronous=OFF`, `locking_mode=EXCLUSIVE`, 4 KiB pages, no mmap, `busy_timeout=0`) and holds only changes (Table 13). All tables are `STRICT`; all except the payload table and two single-key tables are `WITHOUT ROWID`. `fs_data` keeps a rowid so live runs can be overwritten in place with incremental blob I/O.

Table	Rows
`fs_inode(inode, generation)`	One row per changed inode per generation: size, mode, mtime, kind, `inherited_cutoff`
`fs_entry(parent, name, generation)`	One row per changed name; a NULL inode is a whiteout
`fs_data`	File bytes as 4 KiB cells with a per-byte validity mask, or dense runs of whole cells inside one aligned 32 KiB slot
`fs_data_ref`	*Reference pieces*: ranges declared equal to committed bytes of a content root; no bytes stored
`fs_truncate`	A shrink "staircase" so stale cells can be identified by epoch
`fs_predecessor`	The file a rename replaced; an advisory delta base for the next Commit
`fs_digest`, `cleanup_task`	Append-only BLAKE3 digests; the reclamation queue

Table 13: Overlay schema (selected tables).

Layered view. A Workspace's view is the committed base root plus overlay rows with $\mathit{installed} < \mathit{generation} \leq \mathit{active}$. Each inode row carries an `inherited_cutoff`: bytes below it not written in that generation fall through to the next lower layer and finally to the base file. Reads compose at most four local layers newest-first and return the locally decided bytes, an *inherited* bitmask with one covering span, and reference ranges; the request then reads the span and ranges from the immutable Store *after* releasing the Workspace lock.

Request path. All state of one Workspace sits behind one mutex. A job holds it for at most one transaction and performs no Store I/O; base facts a job lacks are returned as *needs*, read without the lock, and supplied to a second job that decides over the then-current rows. Kernel lookup counts, open descriptors, directory cookies and read windows live in in-memory maps that commit or roll back with the job's SQL. Windows bound each job, not files: a FUSE WRITE carries at most 128 KiB (the negotiated `max_write`; the 256 KiB job window is the NFS transfer size), a read at most 128 KiB, a page of rows at most 1,024. Algorithms 1 and 2 show WRITE and READ.

No copy-up. The first write to a committed file inserts one inode row whose cutoff is the base length and stores only the written cells; no base byte is read or copied, and partially written cells keep unwritten bytes inherited through their masks. A truncate edits at most one 32 KiB row, adds one staircase step and lowers the cutoff. Appends past the old length skip even the row-shape read. Renames, including directory renames and rename-over, change at most four inode rows and two name rows and move no bytes; a rename-over records the replaced file as a predecessor. Hard links share one serial and hence one kernel inode.

Capture and install. A Commit seals the active generation by an in-memory state change: *active* becomes *captured* and new mutations go to $\mathit{active} + 1$; no row is copied. Capture waits until every earlier mutation's reply has been attempted, and later mutations wait behind it, so the captured frontier is finite. Construction reads only the sealed generation, in pages of at most 1,024 rows. After publication, install is one SQL step that sets the new base root and raises the floor to the captured generation, plus a pointer swap of the base view; no tree is walked. Rows written after the capture remain on top of the view, because

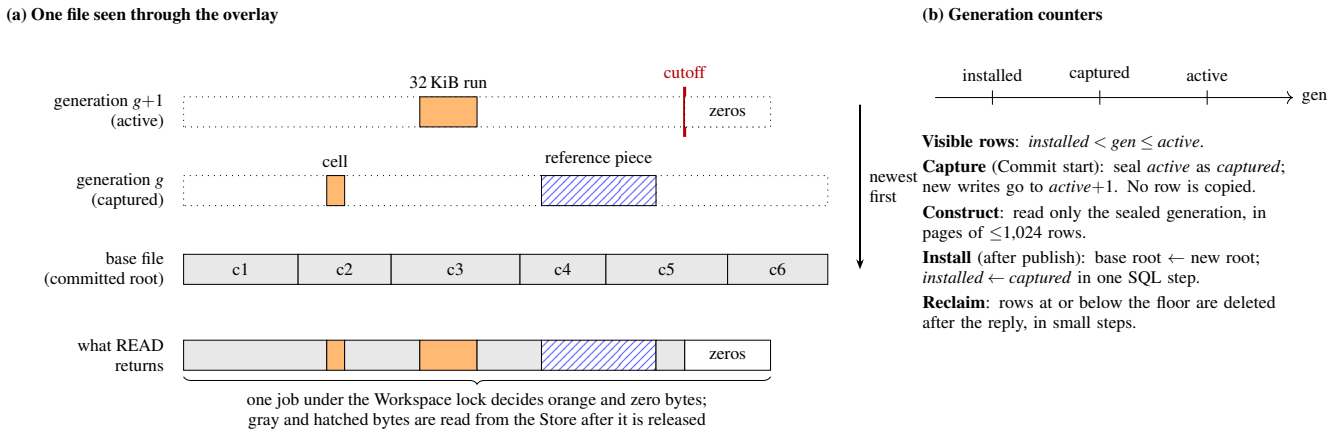


Figure 12: (a) A file seen through the overlay: two generations of rows over the committed file. A READ composes them newest first, deciding locally written and zero bytes under the Workspace lock and reading inherited and referenced bytes from the Store after releasing it. (b) Capture and install move counters, not rows.

Algorithm 1 FUSE WRITE(s, off, d), $|d| \le 128$ KiB

```
claims ← RECOGNIZE(s, off, d)                                   ▷ optional, bounded, no lock
loop
    lock(W)                                                     ▷ one job, at most one transaction
    o ← newest row of s with generation in (installed, active], else base facts
    if o unknown then unlock; fetch base inode of s without the lock; continue
    end if
    begin; if no row (s, active), insert one with cutoff ← o.size
    for all 32 KiB slots overlapping [off, off + |d|) do
        whole cells: overwrite the live dense run in place, or replace it with one run
        partial cell: point read, merge, set validity bits      ▷ no base read
    end for
    cells in claims → reference pieces (no bytes)
    size ← max(o.size, off + |d|); revision += 1; commit; unlock
    reply |d|; return
end loop
```

their cutoffs were taken from the sealed layer that the new root reproduces. Rows of the retired generation are deleted after the Commit replies, in steps of at most 14 cells or 64 rows.

Limits. Kernel write-back caching is off, so each `write(2)` is a synchronous FUSE WRITE published to the overlay, and dirty shared-mapping pages not yet sent are outside a capture. FLUSH and FSYNC answer `ENOSYS` once: durability exists only at Commit. Extended attributes, READDIRPLUS, remote locks, `fallocate` and `copy_file_range` are not implemented. Per-Workspace open state (lookups, descriptors, readers) is the one resident structure without a cap.

D From `write(2)` to Stored Bytes

LayerFS turns filesystem calls into Store objects in two phases. At write time it records *what changed*, cheaply and without reading committed data. At Commit time it turns that record into the smallest set of new immutable objects it can find in bounded work. This appendix follows one byte range through content addressing, chunking, deduplication, compression and delta encoding. Measurements are the evaluation's single samples (Section 8); stored bytes are exact.

Write time: localized-edit capture. Kernel write-back is off, so every `write(2)` reaches the daemon and is published into the Workspace's overlay (Appendix C) before it returns. Bytes are kept as 4 KiB cells, whose per-byte validity mask distinguishes a written zero from a hole, or as dense runs of up to 32 KiB. Holes cost no rows, a shrink is one staircase row, and a write never reads the committed bytes it partly covers. Two kinds of provenance are kept beside the bytes. A *reference piece* says that a

Algorithm 2 FUSE READ(s, off, n), $n \le 128$ KiB

lock(W) ▷ read-only job
$L \leftarrow$ newest ≤ 4 rows of s with generation in $(installed, active]$
for all $\ell \in L$, newest first **do**
    copy live cells and runs of ℓ into undecided bytes
    mark bytes covered by ℓ's reference pieces as reference ranges
    mark bytes $\ge \ell.cutoff$ below EOF decided (zero if unwritten)
end for
$r \leftarrow$ base root at this instant; unlock
if every byte decided **then** reply
end if
read the inherited span of the base file at r and the reference ranges ▷ outside the lock
reply

Operation	Overlay cost	Base cost	In tree size	In file size
Mount	fixed template copy	one root object	$O(1)$	–
LOOKUP, GETATTR	$O(\log R)$	$O(\log D + \log N)$ pages if unchanged	logarithmic	$O(1)$
READ w	$O(w + L \log R)$	$O(w) + O(\log F)$ mapping pages	$O(1)$	logarithmic
WRITE w	$O(w + \log R)$	none	$O(1)$	$O(1)$
Truncate	one $\le$32 KiB row $+O(\log h)$	none	$O(1)$	$O(1)$
CREATE, UNLINK, RENAME, LINK	$\le$4 inode + $\le$2 name rows	name facts	$O(1)$ rows	$O(1)$
Capture, install	in memory / one SQL step	one root object	$O(1)$	$O(1)$

Table 14: Per-request cost. R changed rows, N inodes, D directory entries, F file size, $L \le 4$ layers, h staircase height. Logarithmic base costs follow from the B$^+$-tree layout.

range equals bytes of a committed file: *write recognition* creates one when a write of at least 8 KiB contains whole chunks the file already had, as when a program rewrites a file in place, and stops after 1 MiB of unmatched bytes. A *predecessor* row records that `rename()` put one file over another, which is how editors, Git and package managers save.

Capture to edit list. A Commit seals the overlay generation without copying rows. For each changed file a first pass reads only row *shapes*: an unchanged region is one indexed gap, so the pass grows with rows written, not with file length. A one-pass script turns the spans into an ordered edit list in result coordinates. Bytes inherited from the file's own base, or reference pieces of it, are *retained*; everything else is new. Overwrites, appends (one trailing insert), truncations (one trailing delete), holes (zero runs) and shifted rewrites (pieces plus a small residue, with up to 64 KiB at each edge compared against the base and trimmed where equal) all come out in this one form, and a rewrite that reproduces the base exactly returns the base root.

Construction. Content applies the edits to the stored extent tree (Algorithm 3): it splits extents at each boundary, drops the replaced middle unread, chunks only the replacement, and rebuilds pages only on the join paths. Unchanged extents keep their chunk identities, possibly as slices. A file the base did not hold—a new path, a renamed-over temporary file, a recreated path—is built from scratch; because chunking is content-defined, its unchanged chunks resynchronize and are the same objects as before. *Small files* below the per-Store cutoff (128 KiB by default; any power of two up to 1 MiB) are single whole-file objects, rebuilt whole on every edit, so their natural delta base is the previous version of the whole file.

Storage. Each object's identity is the BLAKE3 hash of its canonical bytes, so identical files, chunks, pages and subtrees are stored once across Commits, Branches and projects. The Save reuses any object already located, then gives each new payload one representation trial (Algorithm 4). The independent record is a level-3 zstd frame, or raw when a 1 KiB probe saves at most 1/32. At most one delta base is tried, and the delta is kept only if strictly smaller. Bases come from filesystem events rather than search: the edited version, the aligned old chunk, the renamed-over file or the file previously at the same path; a similarity sketch is the fallback for whole files only. Tree pages get COPY/INSERT programs against the page they replace. Chains are bounded in depth and decode work, and a chain that reaches a bound is re-based onto its root. Records are grouped into append-only 256 KiB packs inside the single Store database; the ordinary lane compresses tree-page groups with zstd level 1.

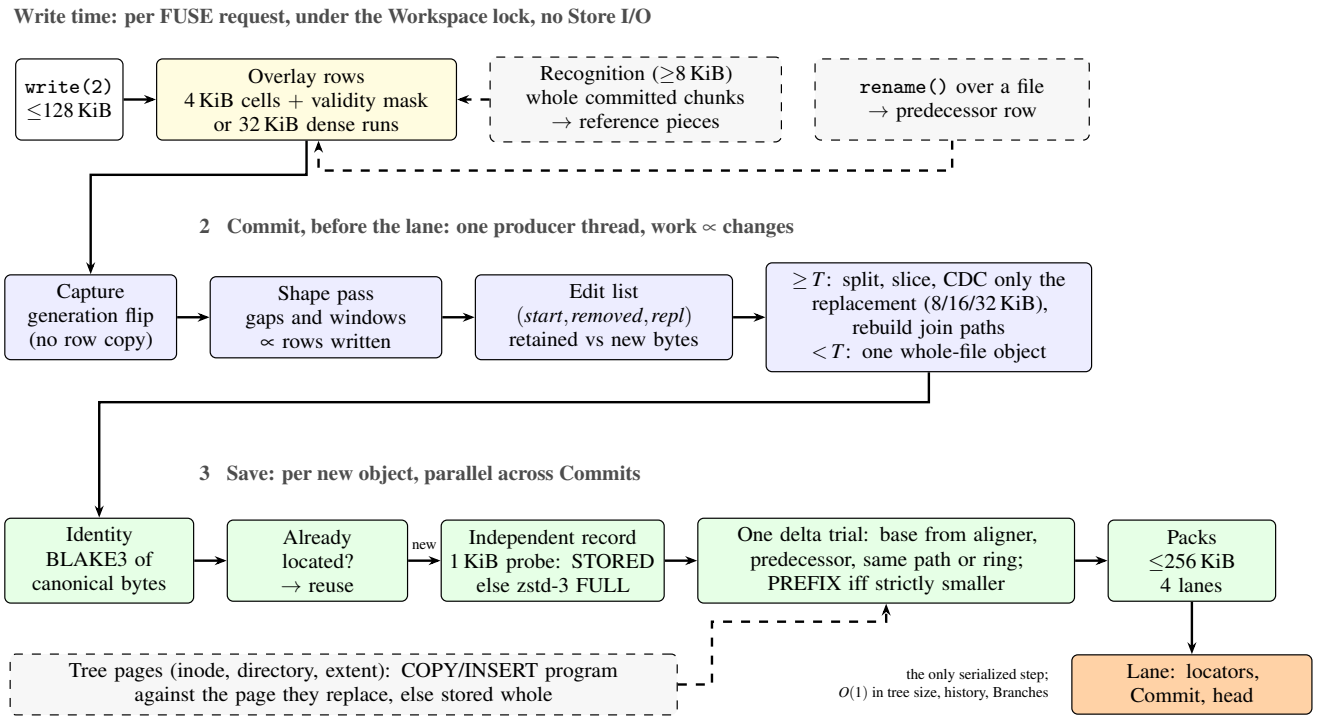


Figure 13: From `write(2)` to stored bytes. Row 1 runs per request without Store I/O; rows 2 and 3 run before the publication lane, in parallel across Commits; only the lane is serialized.

The codec is frozen deliberately. An offline study of 9,386 files under 128 KiB found zstd-6 4.4% smaller than zstd-3 at 2.6× the compression CPU, a trained dictionary 14% smaller but requiring a new frame format, and 256 KiB group compression 22% smaller but forcing 256 KiB of decoding to serve a 2 KB file. LayerFS keeps per-object, random-access records and spends its effort on base selection and metadata instead. The cost is visible: per-chunk frames lose cross-chunk context, so 64 MiB of synthetic text stores 1.25× larger than a whole-file `zstd -3`.

Example 1: a 100-byte overwrite in a 500 MiB file. The write produces one cell row in the overlay (two if it crosses a 4 KiB boundary) and reads nothing from the Store; recognition is skipped below 8 KiB. At Commit the shape pass finds one local window between two inherited gaps and emits a single overwrite edit. The edit reads the mapping pages on the path to that offset—about 27,000 extents in a three-level tree, by calculation—and the one stored chunk under the range, finds that the bytes differ, splits the chunk's extent into two slices, scans the 100 bytes into one new chunk, and rebuilds only that leaf, its ancestors and the 93-byte file state. Each rebuilt page is offered its predecessor as a COPY/INSERT base, and the old overlapping chunk is offered as the new chunk's delta base (random data stays raw). The parent directory is untouched, because directory entries bind inode serials rather than content. Over five Commits each stored 153 B of payload and 5.7 KB of metadata; the Store grew 7.4 KB per Commit and a cold Commit took 28 ms, against 524 MB and 0.6 s for a whole-file copy-up overlay. The same edit on 10 MiB and 100 MiB files stores the same 153 B; in a unit test, a larger file adds one page read per tree level (3 pages at 8 MB, 4 at 24 MB, the same three payloads).

Example 2: a rename-over save of a small file. An editor writes `f.tmp` (a new inode, say 40 KiB) and calls `rename(f.tmp, f)`. The overlay stores the new bytes as dense runs, rebinds the name `f` to the new serial and records the old serial as its predecessor; no Store I/O happens. At Commit the new file has no base of its own, so it is built from scratch as one whole-file object, and the old file's root is declared as an explicit delta base without being read. The Save tries one zstd frame with the old version as prefix (the 256 KiB window covers both) and keeps it if smaller. The directory page that maps `f` gets a COPY/INSERT program against the page it replaces. On 100 such files with 1% line edits, 20 Commits stored about 613 B per file version (32.0×, against 32.1× for in-place editing); on a smaller suite of 25 files the rename-over and in-place payloads are byte-identical. For a large file saved this way, such as Git's index, the new file is re-chunked in full, unchanged chunks dedupe by identity, and the aligner offers each new chunk the old chunk at the same position: index churn fell 80%, with 110 of 120 chunks stored as deltas.

Workload pattern	Absorbed by	Measured
Copy of a 50 MB tree	File-digest relink, content addressing	Copy Commit of 500 MB: 15.9 KB, no payload; 11.9× over 10 copies
Small text edit in place	Whole-file PREFIX delta against the edited version	delta+zstd 32.1× (100 files, 20 Commits of 1% edits)
Rename-over save, small file	Predecessor recorded at `rename()`	32.0×; payload equal to in-place editing
Rename-over save, large file	Content addressing + aligned chunk bases	Git index churn −80% (1.46 MB → 0.30 MB)
New path per version	Same-path base, similarity ring	30.1×
100 B overwrite, 500 MiB file	Cell row, one edit, one new chunk	153 B payload, 5.9 KB stored per Commit
100 B insert shifting a 10 MiB tail	Recognition → reference pieces	153 B payload, 4.0 KB metadata per Commit
16 KiB append, 5 MiB file	Trailing insert, losing-base back-off	17.0 KB per Commit (+0.3% payload)
Truncate 500 MiB to 0	Split and discard	8.4 KB, metadata only
500 MiB of random data	Raw after probe	+0.11% payload
64 MiB of zeros	Zero runs, repeated pages	76 B payload, 684 B metadata
SQLite page churn	In-place page edits, chunk deltas	13.7× raw÷stored (a Git pack is 2.3× smaller)
Tiny files	Whole-file objects, pooled metadata	0.91×: Store larger than raw
20 projects in one Store	Store-wide content addressing	16.7× smaller than 20 Stores

Table 15: Which mechanism absorbs which workload pattern.

Costs. Tiny files cost about 200 B each beyond their bytes, so a Store of 2 KB files is about 10% larger than the raw data. Metadata is about two-thirds of a typical small Commit; fitted per-operation costs are 414 B per edit, 1,360 B per new file, 1,111 B per delete and 464 B per rename. New data is written twice, once to the overlay and again to the Store, so a 500 MiB random write commits more slowly than in a copy-up overlay (1.36 s against 1.0 s cold). A large file saved under a new inode is re-chunked in full: storage stays small, but CPU does not, because recognition covers only same-inode writes. Edits fragment the mapping into slices with no defragmentation, and equal bytes may then have different roots, which is why diff and merge compare bytes. Nothing is garbage-collected or repacked: removing a 500 MiB file frees nothing until a collector exists.

E Algorithms

E.1 Content-defined chunking

LayerFS uses a FastCDC-style GEAR scanner [36] with minimum 8 KiB, target 16 KiB and maximum 32 KiB, two bytes per step, and normalization level 2: the cut probability per byte is 2^{-16} between 8 and 16 KiB and 2^{-12} between 16 and 32 KiB. The first 8 KiB of each chunk are copied without hashing. Scanner state (chunk buffer, hash, and a pending odd byte) persists across reads, so read-buffer boundaries never move a cut. The profile (sizes, masks, seed and table) is digested into every file state.

Zero runs. For a run of zeros the scanner state repeats with period 32 KiB. A hole of length ℓ is handled as one 32 KiB scan, a repeat count $n = \lfloor(\ell - 32\,\text{KiB})/32\,\text{KiB}\rfloor$ of one all-zero chunk, and the remainder; the mapping builder emits *one* uniform page of 128 identical entries to stand for every full page and recurses on the next level. A hole of any length therefore costs $O(32\,\text{KiB})$ scanning plus $O(h \cdot 128)$ mapping work for tree height $h = O(\log_{128}(\ell/32\,\text{KiB}))$, and yields the same partition and root as byte-by-byte scanning.

E.2 Localized edits and the aligner

Edits are $(\mathit{start}, \mathit{removed}, \mathit{replacement})$ in result coordinates. Algorithm 3 applies them to a chunked file. The removed middle is discarded unread; only replacement bytes are chunked, with a fresh scanner, so CDC never resynchronizes across an edit boundary and the two boundary chunks become slices. Each edit costs two splits and two concatenations, each touching $O(h)$ pages on one root-to-boundary path. The cost is therefore independent of file size: the same overwrite reads 3 mapping pages on an 8 MB file and 4 on a 24 MB file.

E.3 Choosing a representation

Each new payload object first receives an independent record: `STORED` if a 1 KiB zstd probe saves at most 1/32, else a level-3 zstd frame (falling back to `STORED` if the frame is not smaller). Algorithm 4 then makes at most one delta trial. Candidates come from filesystem events first: the aligner for edited files; for fresh files, the rename-over predecessor recorded by the overlay, else the file the base held at the same path. Only whole files with no event-derived base consult the similarity ring. T is the Store's whole-file cutoff (128 KiB by default).

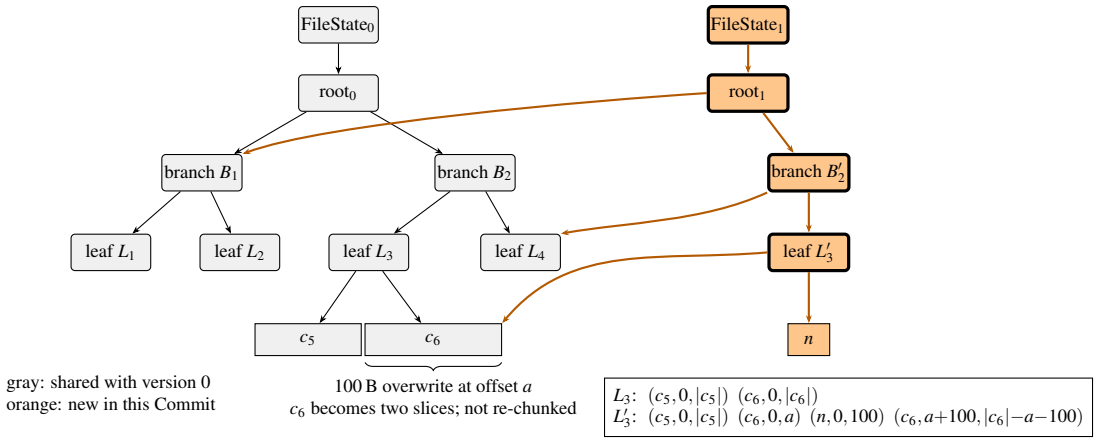


Figure 14: A 100 B overwrite inside chunk c_6. The Commit writes only the orange path (leaf, branch, root, file state) and one 100 B chunk n; c_6 is kept as two slices, and every other page and chunk is shared with version 0.

Algorithm 3 Apply localized edits to a chunked file

function REPLACECHUNKED($base, E$)
  $T \leftarrow$ mapping tree of $base$; $A \leftarrow$ ALIGNER($base$); $shift \leftarrow 0$
  for all $e \in E$ in ascending order **do**
    $(L, R_0) \leftarrow$ SPLIT($T, e.start$); $(X, R) \leftarrow$ SPLIT($R_0, e.removed$) ▷ X discarded unread
    $M \leftarrow \perp$
    if $e.replacement > 0$ **then**
      A.anchor($e.start$, $e.start - shift$)
      $M \leftarrow$ CDC-build(replacement bytes, delta hint $\leftarrow A$.offer)
    end if
    $shift \mathrel{+}= e.replacement - e.removed$
    $T \leftarrow$ CONCAT(CONCAT(L, M), R) ▷ rebuild join paths only
  end for
  return new file state over T
end function

function OFFER($start, len, id$) ▷ aligner: delta base for a new chunk
  $[f, t) \leftarrow$ old byte range $[start - shift, start - shift + len)$
  if an old extent in the window has payload id **then**
    re-anchor $shift$ at it; **return** $\perp$ ▷ unchanged chunk: exact reuse
  end if
  return the old chunk with the largest overlap with $[f, t)$, else the successor of the last anchor
end function

Back-off. A chunk base whose trials keep losing—for example the old last chunk offered to every appended chunk—is skipped starting at the eighth consecutive loss, with the skip doubling per further loss up to 2^{16}; a win forgets it. On a 500 MiB append this reduced delta trials from 26,966 to 22 with identical stored bytes.

Similarity ring. A whole file's signature is the bottom-8 of a splitmix-finalized Rabin–Karp hash over 16-byte windows [8], folded to 32 bits. The ring holds 8,192 slots and a 65,536-entry reference table (704 KiB in total, persisted with the Store). A proposal requires at least two of the eight hashes to overlap and prefers the newest entry, so the latest version of a lineage is proposed. Only objects that found no other base, or won against a ring-proposed base admitted before the current Save, enter the ring. A false proposal costs one trial and is harmless under the strictly-smaller rule.

E.4 Exact reuse and chain reads

Objects reach storage in waves. Duplicate identifiers within a wave are compared byte for byte. An identifier already located with the same role and canonical length is reused without reading it back; integrity is deferred to the read path. Concurrent writers are arbitrated by insert-if-absent locators, and a losing writer compares its bytes with the winner's (a mismatch is a collision error). Reading an object resolves its delta chain iteratively, rejects cycles and chains longer than the role's cap, enforces the decoded and encoded budgets, reconstructs base first, and authenticates every element against its identifier.

Algorithm 4 Representation selection for one payload object

function SELECT($id, raw, role, advisory$)
  $full \leftarrow$ INDEPENDENT(raw); $cap \leftarrow 16$ (whole file) or 8 (chunk)
  if $role =$ chunk **then**
    $c \leftarrow advisory[0]$; **if** c is in back-off **then return** $full$
  else
    $c \leftarrow$ first eligible advisory base, else RING.PROPOSE(id, signature(raw))
  end if
  if $c = \perp$ or ineligible **then return** $full$
  end if
  if c is at the depth cap or its chain exceeds the budgets **then**
    $c \leftarrow$ root of c's chain ▷ re-base: new object lands at depth 1
  end if
  if $\Sigma\text{decoded}(c) + |raw| > 4T$ or $\Sigma\text{encoded}(c) + |raw| > 2T$ **then return** $full$
  end if
  $b \leftarrow$ ACQUIRE(c) ▷ authenticated, charged to the budgets
  $p \leftarrow$ zstd-3(raw, prefix = payload(b)) ▷ exactly one trial
  if $role =$ chunk **then** record a win for c iff p saves at least $|full|/8$
  end if
  if $|p| < |full|$ **then return** `PREFIX`(c, p) ▷ strictly smaller; ties keep FULL
  end if
  return $full$
end function

E.5 Commit

Algorithm 5 Commit of a Workspace

function COMMIT(ws,$force$)
  if another Commit of ws is in flight **then return** `Busy`
  end if
  if no dirty rows, no pending merge, and head and base unchanged since bind **then return** `UpToDate`
  end if
  $G \leftarrow$ CAPTURE ▷ seal generation; no row copied
  producer thread: construct changed inodes of G over the immutable base $\rightarrow$ bounded queue (32 batches) $\rightarrow$ Save
  $cur \leftarrow$ head; **if** cur moved and $\neg force$ **then** $root \leftarrow$ MERGE(bound root, candidate, cur)
  Save.finish; validate; prepare install
  acquire publication lane
    $now \leftarrow$ head; **if** $now \neq cur$ **then** $root \leftarrow$ MERGE(cur, $root$, now) bounded to 1,024 inodes
    if conflicted: install merged tree with markers as local base; publish nothing
    else one write transaction: check expected head, insert Commit, advance Branch
  release lane; install new base; retire G after the reply
end function

Capture is $O(1)$. Construction and Save work is proportional to the captured changes. The lane holds two head reads, an optional merge of at most 1,024 changed inodes with byte comparison capped at 1 MiB per file, and one write transaction, independent of tree size, history length and Branch count.

E.6 Three-way merge and byte equality

Merge walks the inode-table difference between the base and one side in windows of 256 *serials*, not paths, so a move on one side and an edit on the other compose (the edit lands at the new path). For an inode changed on both sides it decides in order: identical change or one side unchanged; a `/.lww` rule (newer modification time wins, ours on a tie); a three-way line merge for text files up to 1 MiB, with Git-compatible marker blocks per conflicting hunk; ours for binaries (recorded as a conflict); kept side wrapped in deletion markers for modify/delete; and a type conflict otherwise. The inode-table difference reads only pages whose child identifiers differ.

Because localized edits make a file's root depend on its edit history, equal bytes can have different roots, so "same" means same bytes. Byte equality walks both files' extent cursors, skips extents that are identical in chunk, offset, length and position without reading them, and compares only the remaining ranges in 256 KiB windows with early exit.

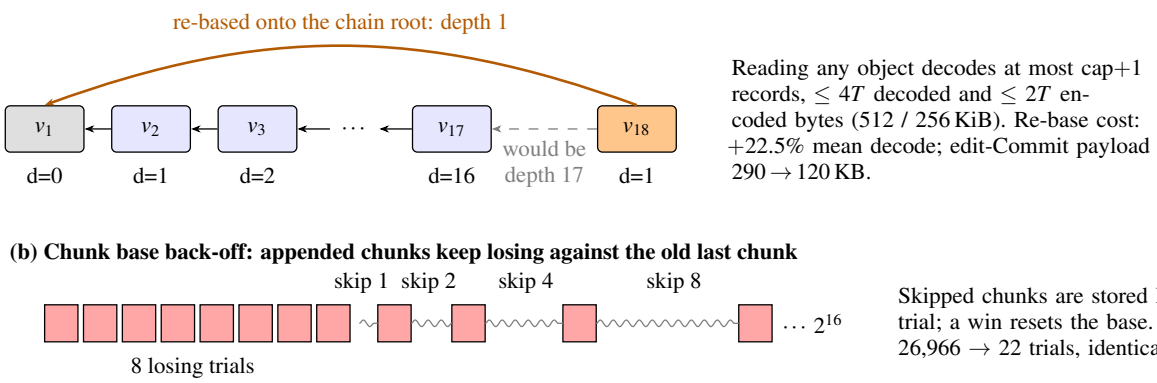


Figure 15: Bounding delta work. (a) A whole-file object that would exceed the depth cap is delta-encoded against the chain root instead of being stored FULL. (b) A chunk base that keeps losing is skipped for exponentially growing gaps.

F Time and Storage Analysis

This appendix derives the time and storage cost of each operation from the structures in Appendices B–E and checks each bound against measurements. Measurements come from the evaluation's campaigns (Section 8). Each is one sample unless noted; storage bytes are exact and times are wall clock.

Notation. A tree has N inodes. A file has length F and $C \approx F/\bar{c}$ chunks, with measured mean chunk size $\bar{c} \approx 19$ KB. Pages hold at most $P = 8$ KiB. The mapping tree of a file has height $h_F = \lceil \log_{64} C \rceil$ or less (3 for 500 MiB), the inode table $h_N = O(\log_{50} N)$, and a directory with D entries $h_D = O(\log D)$. A Commit captures a set Δ of changed inodes; changed file i received w_i written bytes in e_i separate edit runs. T is the whole-file cutoff (128 KiB). H is history length and B the number of Branches.

F.1 Time

Operation	Cost	Grows with	Measured (median)
Mount	Template copy + one root object	nothing	17–43 ms across machines
WRITE w	$O(w + \log R)$ overlay rows; no Store I/O	w	–
READ w, cold	$O(w + h_F + h_N)$ pages; ≤ cap+1 records and ≤ $4T$ decoded per object	w; $\log F$, $\log N$	158 MB/s large file; flat across history (<1%)
Capture	Generation flip	nothing	–
Commit, construction	$O\left(\sum_{i\in\Delta}(w_i + e_i h_F P) + \lvert\Delta\rvert h_N P\right)$	changes only	100 B edit: 24.4 / 24.5 / 28.1 ms at 10 / 100 / 500 MiB
Commit, lane	2 head reads, ≤1,024-inode merge, one transaction	nothing (N, H, B)	no-change Commit 15–21 ms
Commit, new data	$O(w)$ chunking, hashing, zstd, packing	w	500 MiB: 1.36 s (≈385 MB/s)
Fork	One Branch row	nothing	–
Merge	Pages whose child ids differ, windows of 256 serials; ≤1 MiB byte compare per text file	changed serials	–
Unmount	Drain, close, delete overlay file	open state	14–31 ms

Table 16: Time per operation. Mount, Commit and unmount times are CLI calls including process launch and routing, which costs 26–54 ms alone; the Commit times are cold.

File size does not enter a Commit. A localized edit touches one root-to-boundary path per edit boundary, so its construction cost is $O(w + e h_F P)$, and h_F grows by one level per 64–128× growth in F. A 50× increase in file size (10 to 500 MiB) raised

the cold Commit of a 100 B overwrite from 24.4 to 28.1 ms; a copy-up overlay grew from 191 to 604 ms over the same range, and its time is $\Theta(F)$. In a unit test the same three-extent overwrite read 3 mapping pages on an 8 MB file and 4 on a 24 MB file.

Tree size, history and Branch count do not enter the lane. The publication lane holds two head reads, a merge bounded to 1,024 inodes, and one transaction that inserts a Commit and advances one Branch head. None of these reads the tree beyond the bounded merge, walks history or enumerates Branches. Work that does grow—capture of large changes, construction, compression, pack writes, and a full merge when the head has moved—runs before the lane, so concurrent Commits overlap in all of it (Figure 10).

Reads are bounded by structure, not history. A cold read of w bytes costs $O(w)$ plus one root-to-leaf path in the inode table and in the file's mapping tree. Each stored object resolves a delta chain of at most 16 (whole file) or 8 (chunk) records, decoding at most $4T = 512$ KiB and reading at most $2T = 256$ KiB encoded; a chain that would exceed these bounds is re-based onto its root when written. Reads of a 65 MB tree and of a 500 MiB file were within 1% of each other before and after 40 and 5 Commits of history. Large-file reads are slow in absolute terms (158 MB/s cold against 943 MB/s for ovnfs) because each 128 KiB FUSE request is a round trip with one background request outstanding; that is a kernel-route cost, not a history cost.

Per-call overhead. For per-tool-call Workspaces, the cost added to each call is mount, Commit and unmount. On a Docker machine with a SQLite Store the clean tiny-workload row measured 19.2, 16.0 and 16.8 ms, about 52 ms in all with process launch included. Sandbox creation is separate and amortized over many calls: 0.39 s median on Docker, 44–68 ms natively on macOS and in Apple containers. File calls that need no process pay neither, because they run in the SDK's process (Section 4.1).

Where time is lost. New data is written twice, once into the overlay and again into the Store, so a 500 MiB random write committed in 1.36 s against 1.0 s for a copy-up overlay. FUSE publication of every `write(2)` makes write bandwidth a small fraction of a kernel overlay's (Appendix G). Large files saved under a new inode are re-chunked in full at Commit.

F.2 Storage

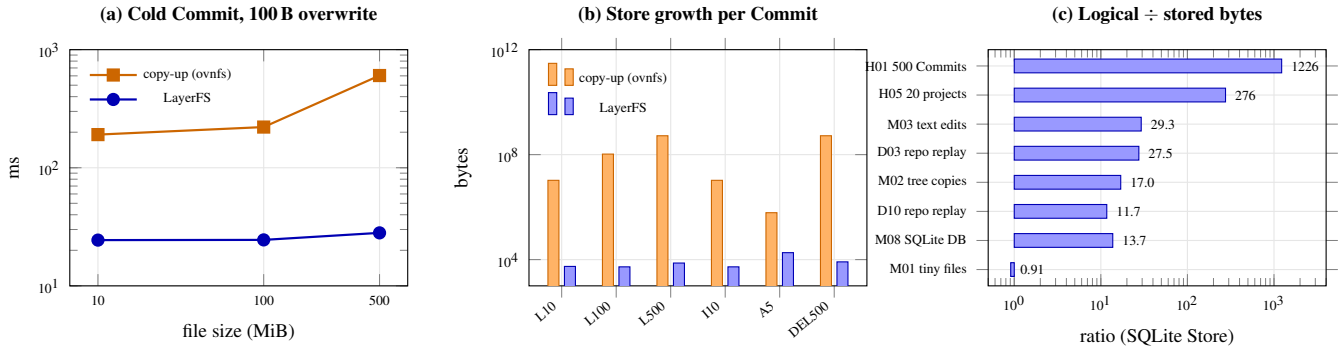


Figure 16: Measured time and storage. (a) Cold Commit time of a 100 B overwrite is flat in file size for LayerFS and grows with it for a copy-up overlay. (b) Store growth per Commit for large-file edits (log scale; DEL500 truncates 500 MiB). (c) Logical snapshot bytes over stored bytes across workloads; tiny files are the one case where the Store is larger than the data.

Cost of one change. Store growth from a Commit is new payload plus new metadata plus pack and locator overhead. Unchanged files, chunks, pages and subtrees cost nothing, because identities are content hashes and updates rewrite only paths to changes.

- *Payload.* A localized edit of w bytes in a chunked file creates between $\lceil w/32\,\text{KiB} \rceil$ and $\lceil w/8\,\text{KiB} \rceil + 1$ new chunks, because only the replacement is chunked and boundary chunks become slices of existing chunks. Payload is therefore at most w plus per-record framing (or a smaller delta), independent of F. A 100 B random overwrite stored 153 B of payload at 10, 100 and 500 MiB. A small file ($< T$) is rewritten whole, so its payload is a delta against the previous version, bounded by its compressed size.

- *Metadata.* A changed file rewrites at most h_F mapping pages, a 93 B file state, h_N inode-table pages and a 116 B root; a name change also rewrites h_D directory pages. Canonically this is $O((h_F + h_N + h_D)P)$, 17–26 KB for a mid-file overwrite of a large file. Each rewritten page is stored as a COPY/INSERT program against the page it replaces, so 3.7–5.7 KB is stored. Across mixed histories a least-squares fit gives 414 B per edit, 526 B per append, 464 B per rename, 1,111 B per delete and 1,360 B per new file.
- *Store overhead.* SQLite pages, locators and indexes add 1–20% over the stored records (1.01–1.20×; PostgreSQL 1.05–1.39×). Per object the floor is about 200 B, which dominates only for tiny files.

Change	Cost bound	Measured
100 B overwrite of a 500 MiB file	$O(w + h_F P)$	7.4 KB growth per Commit (copy-up overlay: 524.3 MB)
16 KiB append to a 5 MiB file	$O(w)$	18.4 KB per Commit (613 KB)
Truncate 500 MiB off a 1,000 MiB file	$O(h_F P)$	8.2 KB (524.3 MB)
Remove a 500 MiB file	$O(h_N P)$	24.6 KB; frees nothing
Copy a 50 MB tree 10 times	O(metadata)	15.9 KB per copy Commit, 0 payload
Fork a Branch	one row; first Commit pays its paths	5.4 KB per identical Branch
Second project in a shared Store	O(metadata) for shared content	220 KB per Init (first: 23.6 MB)
Commit in a 65 MB tree (5 mixed ops)	$O(\sum$ changes$)$	5.9 KB growth per Commit over 500 Commits

Table 17: Store growth per change. Copy-up figures are ovnfs on the same host.

Cost of a history. Store size after k Commits is the initial tree plus the sum of per-change costs; it does not grow with the size of each snapshot. Over 500 Commits on a 65 MB tree (32.5 GB of logical snapshots) the Store reached 26.5 MB: 23.6 MB of initial tree plus 5.9 KB per Commit, a ratio of 1,226× between logical and stored bytes. Table 18 compares the growth of k snapshots in which each snapshot changes m files of size F by w bytes. Copy-up overlays and whole-file snapshot images grow with F; block copy-on-write grows with the touched blocks, but has no cross-file deduplication or deltas; LayerFS grows with w plus a logarithmic path. A fully repacked Git pack is the tightest point of reference—1.10 to 1.31× smaller on the measured histories, because one global repack deltas across all versions—but Git keeps no live mountable view and must repack to reach that size.

System	Growth for k snapshots	Measured point
Copy-up overlay; whole-file snapshot images	kmF	100 B edits to 100 MiB file: 1.15 GB after 10
Block copy-on-write (Btrfs, ImageFS-blk)	$km\lceil w/b\rceil b$ + metadata	4 KiB change to 512 MiB file: about 36 KiB (Btrfs; recovered report)
Git after full repack	$\approx km\,\delta(w)$, after repack	1.10–1.31× smaller than LayerFS
LayerFS	$k(m(w + c\,h_F P') + h_N P')$	same 10 edits: 106.3 MB, of which 104.9 MB is the initial file

Table 18: Growth over k snapshots. b is the block size, $\delta(w)$ a delta of w bytes, and P' the stored size of a page program.

Trade-offs taken. Several storage choices were made, or refused, on their time cost, following the rule that a large speed loss is not traded for a small storage gain (Table 19).

G Comparison with Image-FUSE and Overlayfs Sandboxes

Agent sandboxes commonly build their root filesystem from a lazily loaded, read-only image and a writable copy-on-write layer. Modal's design is a well-documented example with two generations [19, 7, 18]. *ImageFS* serves images through a FUSE server backed by a whole-file content-addressed cache spanning memory, local SSD, zonal caches, a CDN and blob storage, with a kernel overlayfs upper on XFS above it; whole-file copy-up in that arrangement can, in Modal's words, cost gigabytes of overhead to write individual bytes. For VM sandboxes, Modal reports that exposing this stack over virtio-fs made metadata-heavy work several times slower (recursive `chown` from about 4 s to over a minute) and replaced it with *ImageFS-blk*: the host synthesizes a

Choice	Time effect	Storage effect
Re-base at the chain bound	+22.5% mean decode; max 520,575 B $\leq 4T$ budget	Edit Commit payload 290 KB → 120 KB
Losing-base back-off	500 MiB append: 26,966 → 22 trials, 7.9 → 5.5 s	Identical bytes
Win saving <1/8 counts as loss	100 MiB log append 1,540 → 977 ms	+1.04% stored
zstd-3, not zstd-6 (refused)	zstd-6 costs 2.6× compression CPU	zstd-6 would save 4.4%
Per-object records, no group compression (refused)	Group: decode 256 KiB to serve 2 KB	Group would save 22%
Per-chunk frames	Random-access reads	Text 1.25× larger than whole-file zstd
No garbage collection or repack	No background work	Deleted data is retained

Table 19: Speed and storage trade-offs, from the design records.

sparse ext4 image, serves its blocks lazily over virtio-blk, and records guest writes block by block in a per-VM sparse file with a dirty bitmap, so the guest kernel owns all metadata [19, 26]. Snapshots are exposed as images that store the modified files, with an alpha memory snapshot that also captures processes; filesystem snapshots expire after 30 days by default and cannot be listed [20]. Table 20 compares the designs.

	FUSE image + overlayfs	Block image (ImageFS-blk)	LayerFS
Lower layer	Read-only image, whole-file content addressing	Synthesized sparse ext4 image	Any committed root of any Branch
Lazy-load unit	Whole file; 32 MB read-ahead	Block, mapped to file ranges	Tree page, whole file <128 KiB, 8–32 KiB chunk; authenticated
Write target	overlayfs upper on XFS	Per-VM sparse file, dirty bitmap	Per-Workspace SQLite overlay
Copy-up unit	Whole file	One block	None: 4 KiB cells; references for unchanged chunks
Metadata ops	Copy-up of lower files unless `metacopy`	In guest (`chown` ≈2 µs)	One overlay row; one FUSE round trip
Snapshot output	Image of modified files	Image of modified files (dirty bitmap)	Immutable Commit; chunk dedup and deltas
Snapshot cost	∝ bytes of modified files	Not published	∝ changed paths and bytes
Fork, history, merge	New sandbox from image; no history	Same	Fork is one row; diff, merge, blame
Distribution	Fleet-wide cache tiers	Fleet-wide cache tiers	One Store (one kernel, or PostgreSQL)
Steady-state I/O	Kernel page cache and overlayfs	Guest ext4 and page cache	FUSE; write-back off; 128 KiB requests
POSIX coverage	Full	Full (Docker inside the VM)	No xattrs, FIFOs, locks, `fallocate`

Table 20: Image-based sandbox filesystems and LayerFS. Modal entries are taken from its publications and documentation, not reproduced.

Where LayerFS wins. The differences that matter for agents are in capture and history. LayerFS never copies up: a 100 B edit to a 500 MiB file commits in 20–28 ms and adds 3–8 KB, whereas a whole-file copy-up overlay (ovnfs, measured on the same host) took about 0.6 s and stored a full copy; across ten such edits to a 100 MiB file LayerFS stores 106 MB against 1.15 GB (Table 5). Against block-level copy-on-write the write-time advantage largely disappears, but the snapshot difference remains: Modal's documented snapshot unit is the modified file, so a database, a Git pack or a build artifact touched by one byte is captured whole, and there is no branch, merge, diff or history. In LayerFS every step is a Commit, forks are one row, and unchanged chunks, pages and subtrees are shared across Commits, Branches and projects.

Where LayerFS loses. Raw I/O and coverage favor the image designs. Every LayerFS write is a FUSE request published to SQLite before it returns, and the mount negotiates 128 KiB read-ahead with one background request. In an earlier three-way measurement on the same host (a v1.0-era build, warm, one sample), kernel overlayfs achieved 6× the sequential-write and 75× the 4 KiB random-write bandwidth of LayerFS and created small files 30× faster; cold sequential reads of a 500 MiB file reach 158 MB/s against 943 MB/s for another FUSE overlay and about 2.5 GiB/s that Modal reports for ImageFS with 32 MB

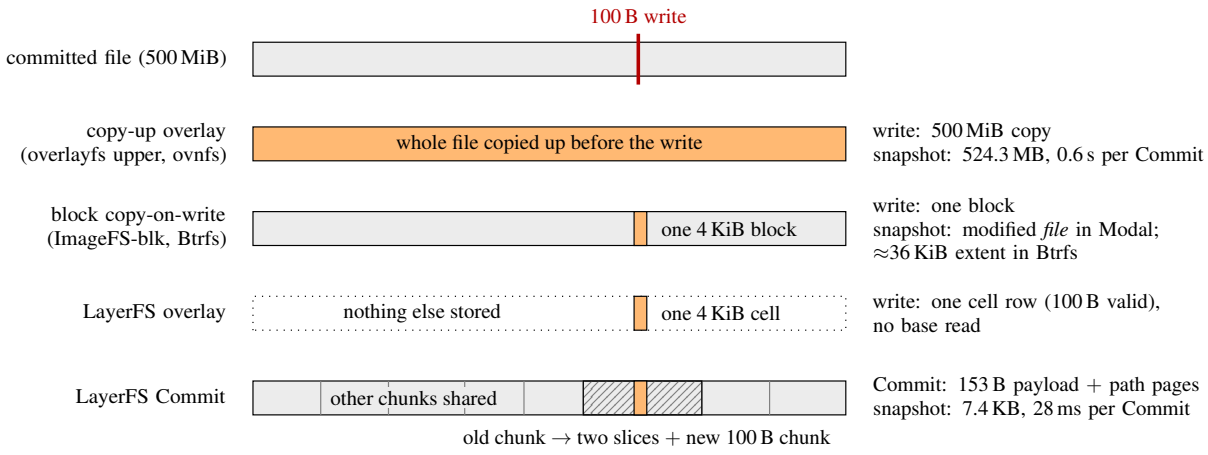


Figure 17: What a 100 B overwrite of a 500 MiB file costs at write time and at snapshot time in each design. Copy-up and Commit figures are measured on one host; block copy-on-write figures are from Modal's description and a recovered Btrfs report.

read-ahead. ImageFS-blk performs metadata operations entirely in the guest. LayerFS omits extended attributes, special files, locks and `fallocate`, so Docker-in-sandbox, which Modal's VM runtime targets, cannot run on a Workspace. Modal distributes images across a fleet, while a LayerFS SQLite Store is confined to one kernel.

Complementary, not competing. The comparison is not a single ordering: LayerFS is better at the operations that make sandbox state versioned—commit cost proportional to change, sub-file copy-on-write, deduplication across snapshots, cheap forks and history—and worse at steady-state throughput, POSIX coverage and fleet-scale distribution. The designs also compose: a LayerFS Workspace can run inside a VM whose block device serves the operating system image, with LayerFS serving only the project. A fair evaluation would run, on one native Linux host, LayerFS, a FUSE-image lower with an overlayfs upper and whole-file snapshots, a block-level copy-on-write analog of ImageFS-blk, and plain ext4, on (1) per-tool-call agent replays with a snapshot after each mutating call, (2) small edits to 10 MiB–1 GiB files, a SQLite database and a large Git repository, (3) Modal's own metadata and install workloads, (4) fork fan-out to 64 views, and (5) 1,000 sequential snapshots, plus (6) a LayerFS ablation of read-ahead, background requests and write-back caching that separates design cost from tuning.

H Command-Line Interface

LayerFS 2.0 exposes one call table: 73 calls in ten families. The `layerfs` command, the Node SDK and the Python SDK are all generated from it, so `layerfs file write` on the command line is `layerfs.file.write` in the SDKs, with the same positional arguments and options. Positionals run from the outside in: owner, Branch, path or new name, other inputs, and an optional Commit last. A project is named `<store>/<project>`. File content comes from standard input or `-from`. Every call prints JSON, or text with `-text`; it exits 0 when done, 2 on a malformed command line and 125 when refused, with a structured `{code, message, detail}` on standard error. Each call is attempted once and never retried. Table 21 lists every call; five more commands exist only on the command line (`help`, `-version`, `doctor`, `import-env` and `worker`).

Table 21: Every call of the LayerFS 2.0 command line, generated from the call table. `[...]` marks an optional positional or option.

Command	What it does
Stores (12)	
`layerfs store attach <name> <machine> [-url <url>] [-password <password>]`	Let a machine run Sandboxes of a PostgreSQL Store, after checking it reaches the server
`layerfs store connect <name> <url> [-password <password>] [-synchronous-commit <synchronous-commit>]`	Attach a PostgreSQL database as a Store
`layerfs store create <name>`	An empty sqlite Store on native
`layerfs store detach <name> <machine>`	Stop a machine running Sandboxes of a PostgreSQL Store
`layerfs store disconnect <name>`	Forget a PostgreSQL Store; its data stays
`layerfs store get <name>`	A Store's record
`layerfs store list`	Every Store, from the home's records
`layerfs store pull <name> [-progress]`	Bring a sqlite Store from its machine to native

Command	What it does
`layerfs store push <name> <machine> [-progress]`	Send a sqlite Store from native to a machine
`layerfs store remove <name>`	Delete a sqlite Store with every project in it
`layerfs store stats <name>`	Store-wide publication and allocation counters
`layerfs store status <name>`	A Store's live state
Projects (6)	
`layerfs project export <project> <destination> [-branch <branch>] [-progress]`	Export a project's snapshot from its Store to a local folder
`layerfs project get <project>`	A project's record: its Store, kind and machine
`layerfs project init <project> <folder> [-timeout <timeout>] [-progress]`	Import a host folder as a new project into an existing Store
`layerfs project list [<store>] [-page-size <page-size>] [-page-pos <page-pos>]`	Every project in the home, or one Store's
`layerfs project remove <project>`	Remove one project from its Store
`layerfs project stats <project> [<branch>] [<commit>]`	Entry counts of one pinned saved tree
Branches (8)	
`layerfs branch commits <project> <branch> [-page <page>] [-page-size <page-size>] [-at-commit <at-commit>] [-include-storage]`	Saved branch commits or main publications, newest first
`layerfs branch diff <project> <from-branch> <to-branch> [-paths-only] [-page-size <page-size>] [-page-pos <page-pos>] [-cursor <cursor>] [-page <page>] [-from-commit <from-commit>] [-to-commit <to-commit>] [-path-prefix <path-prefix>] [-include-attribution] [-continuation <continuation>]`	Files that differ between two Branches' heads
`layerfs branch fork <project> <name> <from-branch> [<from-commit>]`	A new Branch from another Branch's head or Commit
`layerfs branch get <project> <branch>`	One Branch and its head Commit
`layerfs branch get-commit <project> <branch> <commit>`	One saved Commit or main publication
`layerfs branch list <project> [-page-size <page-size>] [-page-pos <page-pos>] [-page <page>]`	A project's Branches
`layerfs branch merge <project> <name> <a> <b> [-message <message>] [-units <units>]`	Merge two Branches into a new one
`layerfs branch publish <project> <branch> [<commit>]`	A Branch's head or Commit as main's next Layer
Files (in-process; no Sandbox) (15)	
`layerfs file apply <project> <branch> [-message <message>] [-from <file>]`	Apply a list of changes; one Commit
`layerfs file blame <project> <branch> <path> [<commit>] [-from-line <from-line>] [-line-count <line-count>]`	The Commit of each line
`layerfs file cp <project> <branch> <from-path> <to-path> [-message <message>]`	Copy a path; one Commit
`layerfs file diff <project> <left-branch> <left-path> <right-branch> <right-path> [-left-commit <left-commit>] [-right-commit <right-commit>] [-include-attribution] [-allow-missing] [-page <page>] [-page-size <page-size>]`	The difference between two files
`layerfs file edit <project> <branch> <path> [-old <old>] [-new <new>] [-message <message>]`	Replace one unique text in a file; one Commit
`layerfs file glob <project> <branch> <path> <pattern> [<commit>] [-page-size <page-size>] [-page <page>] [-page-pos <page-pos>]`	Paths under path matching a glob
`layerfs file grep <project> <branch> <path> <pattern> [<commit>] [-page-size <page-size>] [-page <page>] [-page-pos <page-pos>]`	Lines under path matching a regular expression
`layerfs file list <project> <branch> <path> [<commit>] [-page-size <page-size>] [-page <page>] [-page-pos <page-pos>]`	A folder's entries
`layerfs file mkdir <project> <branch> <path> [-message <message>] [-mode <mode>]`	Make a folder; one Commit
`layerfs file mv <project> <branch> <from-path> <to-path> [-message <message>]`	Move a path; one Commit
`layerfs file read <project> <branch> <path> [<commit>] [-start-line <start-line>] [-end-line <end-line>]`	Read a file
`layerfs file revision <project> <branch> <path> [-at-commit <at-commit>] [-traversal <traversal>] [-page <page>] [-page-size <page-size>] [-continuation <continuation>]`	Saved changes to a literal path
`layerfs file rm <project> <branch> <path> [-message <message>]`	Remove a path; one Commit
`layerfs file stat <project> <branch> <path> [<commit>]`	A path's kind, size and mode
`layerfs file write <project> <branch> <path> [-message <message>] [-mode <mode>] [-from <file>]`	Write a file; one Commit
Commits (3)	
`layerfs commit diff <project> <from-commit> <to-commit> [-paths-only] [-page-size <page-size>] [-page-pos <page-pos>] [-cursor <cursor>]`	Files that differ between two Commits
`layerfs commit get <project> <branch> <commit>`	One Commit of a Branch
`layerfs commit list <project> <branch> [-page-size <page-size>] [-page-pos <page-pos>]`	A Branch's Commits, newest first

Command	What it does
Sandboxes (4)	
`layerfs sandbox close <name> [-force]`	Stop a Sandbox
`layerfs sandbox create <project> <name> [<machine>] [-image <image>] [-progress]`	Start a named Sandbox of a project on a machine (default: its sqlite Store's, or native)
`layerfs sandbox get <name> [-page-size <page-size>] [-page-pos <page-pos>]`	A Sandbox and its Workspaces
`layerfs sandbox list [<project>] [-page-size <page-size>] [-page-pos <page-pos>]`	Every Sandbox in the home, or one project's
Workspaces (6)	
`layerfs workspace commit <ws> [-message <message>] [-force]`	Commit a Workspace's changes to its Branch
`layerfs workspace exec <ws> <command> [-message <message>] [-stream] [-exec-id <exec-id>] [-progress] [-events]`	Run one bash command in a Workspace
`layerfs workspace exec-cancel <ws> <exec-id> [-grace-ms <grace-ms>]`	End a running command: SIGTERM, then SIGKILL after grace_ms
`layerfs workspace mount <sandbox> <branch> [-name <name>]`	Mount a Branch in a Sandbox
`layerfs workspace status <ws>`	A Workspace's changes since its base
`layerfs workspace unmount <ws> [-discard] [-force]`	End a Workspace
Machines (12)	
`layerfs machine create <name> <platform> [-cpus <cpus>] [-memory <memory>] [-disk <disk>] [-image <image>] [-os-image <os-image>] [-progress]`	Create an enabled VM machine (apple-container, lima)
`layerfs machine get <name>`	A machine's record
`layerfs machine images <name>`	The Sandbox images a machine holds, and the host's Docker images it can copy
`layerfs machine list`	Every machine, with what it supports
`layerfs machine load <name> <image> [-progress]`	Copy an image from the host's Docker to a machine without creating a Sandbox
`layerfs machine platforms`	The platforms this host and build support, and what each is missing
`layerfs machine prune <name> [<image>] [-unused]`	Remove a Sandbox image from a machine, or every unused one
`layerfs machine remove <name>`	Delete a VM machine and its disk; one of a removed platform is only forgotten
`layerfs machine start <name> [-progress]`	Start a stopped machine and its worker; a running one answers at once
`layerfs machine status <name>`	A machine's live state, free disk and tool version
`layerfs machine stop <name> [-force]`	Stop a machine's worker and VM; its disks stay
`layerfs machine update <name> [-cpus <cpus>] [-memory <memory>] [-image <image>] [-docker <docker>] [-wsl <wsl>] [-daemon <daemon>]`	Resize a VM, or set a built-in machine's image, Docker socket and daemon
Backups (5)	
`layerfs backup connect <name> <url> [-key <key>] [-region <region>] [-part-size <part-size>]`	Connect an r2 or MinIO bucket
`layerfs backup create <store> <bucket> [<file>] [-progress]`	Upload a sqlite Store to a bucket
`layerfs backup disconnect <name>`	Forget a bucket and this home's links to it
`layerfs backup list [<bucket>]`	Every Store file on the connected buckets
`layerfs backup restore <store> <bucket> [<file>] [-progress]`	Download a backup as a sqlite Store on native
Call log (2)	
`layerfs logs list [-op <op>] [-project <project>] [-client <client>] [-tag <tag>] [-outcome <outcome>] [-since <since>] [-cursor <cursor>] [-limit <limit>]`	The calls made against this home, newest first
`layerfs logs reset`	Clear this home's call log
Command line only (5)	
`layerfs help [<family> [<call>]]`	This list; `help -json` prints the call table
`layerfs -version`	The build's version
`layerfs doctor [-quick | -write-test] [-timeout <duration>]`	Report what is wrong in the home, changing nothing
`layerfs import-env`	Write this environment's 1.1.0 settings as records, once
`layerfs worker`	Serve this machine's worker (inside a machine)